Automated Construction of Semantic Knowledge Graphs from Structured Data

Submitted to the
Albert-Ludwigs-Universität Freiburg

by Surbhi Nair



universität freiburg

Master's Thesis

Submitted on **October 27, 2026**
to the **Albert-Ludwigs-Universität Freiburg**
by **Surbhi Nair** (5973324)

Examined by
Prof. Dr. Hannah Bast
Prof. Dr. Joschka Bödecker

Department of Computer Science
Faculty of Engineering
Albert-Ludwigs-Universität Freiburg

Declaration

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used.

Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I also hereby declare that my thesis has not been prepared for another examination or assignment, either in its entirety or excerpts thereof.

Freiburg, October 27, 2026

Place, date	Surbhi Nair

Statement on AI Usage

Acknowledgements

I would like to sincerely thank Prof. Dr. Hannah Bast for serving as my first examiner and for the opportunity to work on this topic at the Chair of Algorithms and Data Structures.

I am also grateful to Prof. Dr. Joschka Bödecker for kindly agreeing to serve as the second examiner and for his time and support.

I would like to express my sincere gratitude to my supervisor, Sebastian Walter, for proposing the topic of this thesis, for keeping its scope well-defined, and for his continued guidance throughout the course of this work. I am also grateful that I could build on GRASP, which he developed together with Prof. Dr. Hannah Bast, as it forms the foundation of the approach in this thesis.

Further thanks go to the corporate design team at the Albert-Ludwigs-Universität Freiburg and Leander Marius Bürkin for this LaTeX-template.

Abstract

Converting relational data into an RDF knowledge graph (KG) is straightforward with a direct mapping, but the result mirrors how the data is stored rather than what it describes. Turning it into a semantically modeled KG, with domain entities, meaningful classes and predicates, and explicit relationships, still requires a knowledge engineer to understand the domain and the schema and to write the mapping by hand.

In this thesis, we investigate to what extent an LLM agent can automate this step by generating SPARQL CONSTRUCT queries that transform a generic KG, obtained by a direct mapping, into a semantic KG. We extend the agent framework GRASP with a generation agent that explores the generic KG through a SPARQL endpoint, designs the model, and tests each query on the data, and an independent review agent that checks the submitted queries against the data, supported by a mechanically computed source coverage. The queries depend only on the schema, so they can be inspected, corrected, and executed on the complete data.

We compare our approach with a general-purpose coding agent, OpenCode, which receives the same generic KG as a file and uses the same LLM. We evaluate both on eight datasets: IMDb, Mondial, and MusicBrainz, with 7 to 239 tables, and five databases of the BIRD benchmark, which we model with and without competency questions. We evaluate the output on three levels: whether the queries execute on QLever, an LLM judge that scores the model against nine criteria, and a manual analysis with analytical SPARQL queries.

All query sets of our approach execute without changes, while six of the twelve query sets of the baseline required interventions, although the baseline reported them as tested. Without competency questions, our approach scores at least as high as the baseline on all seven datasets where both produced a result, with a mean of 8.5 against 7.6 out of 10 on the BIRD databases, and far fewer major issues. On IMDb, its graph is also simpler and faster to query on every analytical question. Competency questions improve the baseline, but slightly lower the scores of our approach, which narrows its model towards the questions. On MusicBrainz, the largest schema, the score drops to 6.7, with errors in the naming and direction of relationships and in date encoding that a user cannot detect from query results.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 2

2 Related Work **7**
- 2.1 Conventional Approaches . . . 7
  - 2.1.1 Direct Mapping . . . 7
  - 2.1.2 Declarative Mappings . . . 7
- 2.2 LLM-Based Knowledge Graph Construction . . . 8
  - 2.2.1 Direct Triple Extraction . . . 8
  - 2.2.2 Mapping Generation . . . 9
- 2.3 Competency-Question-Driven Modeling . . . 10
- 2.4 Evaluation in Related Work . . . 12

3 Background **13**
- 3.1 SPARQL CONSTRUCT Queries . . . 13
- 3.2 LLM Agents and Tool Calling . . . 16
- 3.3 GRASP . . . 16

4 Approach **18**
- 4.1 Structured Data to Generic KG . . . 19
- 4.2 Preparing the Generic KG . . . 22
- 4.3 Generating CONSTRUCT Queries . . . 22
- 4.4 Baseline . . . 29

5 Evaluation **31**
- 5.1 Datasets . . . 31
  - 5.1.1 IMDb . . . 31
  - 5.1.2 Mondial . . . 32
  - 5.1.3 MusicBrainz . . . 34
  - 5.1.4 BIRD-SQL Databases . . . 35
- 5.2 Setup . . . 37
- 5.3 Results . . . 40
  - 5.3.1 Syntactic Validity . . . 40
  - 5.3.2 LLM-as-a-Judge Scores . . . 41
  - 5.3.3 Qualitative Analysis . . . 45

6 Conclusion **56**

List of Figures **I**

List of Tables **II**

Bibliography **III**

Appendix **VIII**

1 Introduction

A large share of today's data is stored in relational databases and CSV files. Converting such data into RDF knowledge graphs (KGs) makes it reusable and queryable beyond the application it was designed for, but only if the data is modeled well. In this thesis, we investigate to what extent an LLM agent can automate this modeling by generating SPARQL CONSTRUCT queries.

1.1 Motivation

The quality of converting structured data, such as relational databases or CSV files, into an RDF knowledge graph (KG) depends on how the data is modeled. Making structured data queryable with SPARQL is straightforward: the direct mapping from relational data to RDF has been standardized by the W3C [2]. It turns every row into a node and every column into a predicate so the resulting graph reflects how the data is stored rather than what it describes. A semantically modeled KG, in contrast, represents domain concepts as entities, describes them with meaningful classes and properties, and expresses relationships between them using a well-designed ontology.

Building a semantically modeled KG like this requires substantial manual effort and expertise: the ontology is traditionally designed manually with collaboration between domain experts and ontology specialists, often following methodologies that range from classical approaches [11] to agile, industry-oriented frameworks such as LOT [27].

Tools have automated the mechanical parts of the conversion: mapping languages such as RML [10] declare how source data maps to RDF, engines such as Morph-KGC [3] execute these mappings at scale, and Tarql [8] transforms CSV files with hand-written SPARQL CONSTRUCT queries. The central part, however, remains manual: understanding the domain and the source schema, and deciding how the data should be modeled. At scale, this becomes all the more challenging.

Large language models (LLMs) are well-suited for this task. The required modeling decisions depend largely on the meaning of table names, column names, and values, which LLMs can interpret. Recent work confirms that LLMs can indeed be used to convert structured data into KGs.

Existing approaches follow two directions: direct triples extraction by an LLM [14, 26], which is severely limited by the size of the data, or the LLM generates a declarative mapping, for example in RML, which a mapping engine then executes [16, 28]. These

experiments, however, are small in scale and domain-specific, or they rely on a target ontology that is given in advance.

One direction that to our knowledge has not been explored yet, is to let an LLM generate SPARQL CONSTRUCT queries that transform an input KG into a semantically modeled one. This direction combines the advantages of the other two. Like a mapping, a CONSTRUCT query is an explicit artifact, which a human can inspect and correct, and its size depends only on the schema, not on the number of rows. Unlike an RML mapping, it requires no separate engine, since the query can be executed by the same SPARQL engine that hosts the data.

In this thesis, we therefore investigate to what extent an LLM agent can automate the semantic modeling of structured data by generating SPARQL CONSTRUCT queries, for relational datasets of varying schema complexity and without a target ontology specifically provided.

1.2 Problem Statement

The goal of this thesis is to automate the step from the output of a direct mapping or a pre-existing knowledge graph to a semantically modeled one. Before we define this task formally, we illustrate the difference with a simple example.

Figure 1 shows three tables of the IMDb dataset. The table `principals` connects titles to persons and records the role of each person in the column `category`.

title

tconst	titleType	primaryTitle
tt1	movie	Heat

person

nconst	primaryName
nm01	Michael Mann
nm02	Al Pacino

principals

tconst	ordering	nconst	category
tt1	1	nm01	director
tt1	2	nm02	actor

Figure 1: Three tables of the IMDb dataset with selected columns.

In the generic KG shown in Figure 2, every row of `principals` becomes a node of its own, which sits between the movie and the person, and the role of the person is only a string attached to this node instead of being expressed as a predicate. The semantic KG instead connects the movie directly to its director and its actor, with the role expressed by the predicates `imdb:director` and `imdb:actor`. A direct mapping cannot produce such a model, because it would have to derive predicates from the values of a column.

```
<row/title/tt1> a <table/title> ;
  <title/titleType>     "movie" ;
  <title/primaryTitle>  "Heat" .

<row/person/nm01> a <table/person> ;
  <person/primaryName>  "Michael Mann" .
<row/person/nm02> a <table/person> ;
  <person/primaryName>  "Al Pacino" .

<row/principals/tconst=tt1;ordering=1> a <table/principals> ;
  <principals/category>   "director" ;
  <ref/principals/tconst> <row/title/tt1> ;
  <ref/principals/nconst> <row/person/nm01> .

<row/principals/tconst=tt1;ordering=2> a <table/principals> ;
  <principals/category>   "actor" ;
  <ref/principals/tconst> <row/title/tt1> ;
  <ref/principals/nconst> <row/person/nm02> .
```

```
title:tt1 a imdb:Movie ;
  rdfs:label    "Heat" ;
  imdb:director name:nm01 ;
  imdb:actor    name:nm02 .
name:nm01 a imdb:Person ; rdfs:label "Michael Mann" .
name:nm02 a imdb:Person ; rdfs:label "Al Pacino" .
```

Figure 2: The generic KG of the IMDb tables (top) and a semantic KG of the same data (bottom).

The difference becomes visible when the data is queried. To ask who directed *Heat*, a query over the generic KG has to traverse the intermediate node and filter it by the string `"director"`, while a query over the semantic KG follows a single edge.

```
SELECT ?name WHERE {
  ?t <title/primaryTitle> "Heat" .
  ?p <ref/principals/tconst> ?t ;
     <principals/category> "director" ;
     <ref/principals/nconst> ?d .
  ?d <person/primaryName> ?name .
}
```

```
SELECT ?name WHERE {
  ?t a imdb:Movie ;
     rdfs:label "Heat" ;
     imdb:director ?d .
  ?d rdfs:label ?name .
}
```

Figure 3: The query “Who directed *Heat*?” over the generic KG (bottom left) and over the semantic KG (bottom right).

The semantic KG shown in this example is, however, only one of many valid choices. Another model could keep each row of `principals` as a node, but as an entity of the domain: a *credit* of class `imdb:Credit`, which links the movie and the person, and refers to the role as an entity, such as `imdb:Director`, rather than as a string. Unlike the direct edges in Figure 3, this model also preserves the billing order recorded in the column `ordering`, at the cost of slightly longer queries. Both models are reasonable even while they differ slightly in structure. Since no target ontology is given, there is thus no single correct semantic KG, and therefore no reference graph against which a result could be compared. Instead, the quality of a semantic KG has to be judged by criteria that are independent of the chosen vocabulary and structure.

To formalise this problem statement, we first introduce the key terminology used.

- D is the input dataset, which has to be converted into a semantic KG. In this thesis, D is structured data, that is, a relational database or a set of CSV or TSV files.

- G_D is the *generic KG* of D: the RDF graph obtained from D by a direct mapping in the style of the W3C Direct Mapping [2]. Section 4 describes this in detail.

- $C = \{c_1, \ldots, c_m\}$ is an optional set of competency questions: natural-language questions that the semantic KG should be able to answer [13].

- $Q = \{q_1, \ldots, q_k\}$ is a set of SPARQL CONSTRUCT queries. Executing Q on G_D yields the graph
$$Q(G_D) = \bigcup_{i=1}^{k} q_i(G_D).$$

- $G = Q(G_D)$ is the output KG.

Task: Given the generic KG G_D derived from D and, optionally the competency questions C, the task is to find a set of CONSTRUCT queries Q such that $G = Q(G_D)$ is a semantic KG of D. Choosing the classes and predicates of G is part of the task. We judge whether G is a good semantic KG of D by three main requirements:

1. G *models the domain* of D, not the structure of its schema;

2. G is *faithful* to D: it preserves the information of D and adds nothing that D does not claim; and

3. G is *usable*: questions about the data can be answered over G with simple queries.

Criteria: We make each requirement concrete with a set of criteria.

Semantic Modeling:

- *Entity and class modeling:* It concerns selection and granularity. Which real-world entities or domain concepts from the source data become entities in the semantic KG? Are the classes and entities at the right level of granularity, or are they too coarse or too fine?

- *Vocabulary:* Do the classes and predicates of G denote concepts of the domain, instead of repeating the source table and column names? And does each predicate have a single, precise meaning and the appropriate or preferred direction?

- *IRI design:* Are entity IRIs minted from stable identifiers of the source data, in a namespace that reflects the domain, and does the same entity receive the same IRI wherever it is referenced?

- *Relationships:* Where reference information is available, are the relationships of the source domain recoverable from G, without G being a structural replica of the source schema?

Fidelity:

- *Schema coverage:* Is every data element(required tables/columns) of D carried over into G?

- *Soundness:* Is everything in G supported by D, that is, does Q not invent entities, values, or links?

- *Structural integrity:* Is G internally consistent? Every link should point to an entity that is actually described in G, and each real-world entity should appear as exactly one node. For example, the same person should not appear under two different IRIs because two queries mint its IRI differently.

- *Literal typing:* Do literals carry datatypes that match their values?

Usability:

- *Answerability:* Can a natural language question be answered over G with a SPARQL query, does this query return the correct answer, and how complex is it compared to a query over G_D?

Research questions: Based on this task and these criteria, overall we aim to address the following questions from our experiments:

RQ1. Can an LLM agent that explores G_D generate a set of CONSTRUCT queries Q such that $Q(G_D)$ can be considered a good semantic model as per the criteria above?

RQ2. Does the approach proposed in this work yield better semantic modeling than a general-purpose LLM agent that is given the same input KG?

RQ3. Does supplying competency questions as additional input change the resulting modeling, and does it improve it?

RQ4. How does modeling quality change as the size and relational complexity of the schema grow, and what are the characteristic failure modes?

Scope: The input is structured data, that is, relational databases and CSV/TSV files, and not unstructured data such as text. Each dataset comes from a single source; we do not integrate several sources into one KG. The output is a materialized RDF graph, not a virtual graph over the source data and not a labeled property graph. The agent may treat placeholder values as missing, but correcting errors in the source data is not a part of the task. For large datasets, the agent explores a sample of the data rather than the complete data. Section 4 describes this implementation in detail.

2 Related Work

This chapter reviews the related work done on KG construction or conversion of structured data into RDF knowledge graphs starting from conventional approaches to more recent work that uses LLMs.

2.1 Conventional Approaches

2.1.1 Direct Mapping

The simplest conversion of structured data into RDF is a direct mapping. The W3C Direct Mapping [2] defines how a relational database can be converted into RDF without any user input: every table yields a class, every row a subject, and every column a predicate; foreign keys yield links between rows.

$$\text{Source Data}(D) \xrightarrow{\text{direct mapping}} \text{generic KG } (G_D)$$

$$\text{Direct Mapping} = \{\text{Table} \rightarrow \text{Class}, \text{Row} \rightarrow \text{Subject}, \text{Column} \rightarrow \text{Predicate}\}$$

Direct mappings are fully automatic, one can use simple scripts to perform such a conversion, but their output reflects the structure of the source schema rather than the domain it describes. Their main aim is simply to make the data queryable through SPARQL. It does not provide a semantic interpretation of the data. In this thesis, we use a simplified direct mapping only as the first step, to obtain the generic KG G_D (Section 1.2).

2.1.2 Declarative Mappings

To obtain a semantic KG, a user must specify how the source data maps to a target vocabulary. Declarative mapping languages express this mapping as a set of rules. R2RML [9] is the W3C recommendation for mapping relational databases to RDF. RML [10] generalizes R2RML to heterogeneous sources such as CSV, JSON, and XML. Mapping engines such as Morph-KGC [3] execute these mappings and materialize the resulting KG at scale.

$$\text{Source Data}(D) + \text{RML Mapping} \xrightarrow{\text{Mapping Engine}} \text{Semantic KG } (G)$$

$$\text{Generic KG}(G_D) + \text{CONSTRUCT Query} \xrightarrow{\text{SPARQL Engine}} \text{Semantic KG } (G)$$

An alternative is to express the mapping as a SPARQL CONSTRUCT query. Tools like Tarql [8] apply a user-written CONSTRUCT query to each row of a CSV file. SPARQL Anything [4] generalizes this idea: it exposes data in many formats through a generic RDF view, called Facade-X, which the user then transforms into the target vocabulary with SPARQL CONSTRUCT queries.

Asprino et al. [4] argue that mapping languages require users to know the quirks of the source format, the target ontology, and a dedicated mapping syntax at the same time, whereas a generic façade combined with a general-purpose query language only requires knowledge of SPARQL. This approach is the closest predecessor to our work. Our approach follows the same architecture: a generic KG, transformed by a CONSTRUCT query. The difference is that, in SPARQL Anything, the query has to be written by the user, while we let an LLM agent generate it.

In all of these approaches, the mapping itself is written manually. The tools automate the execution of the conversion, but not the modeling decisions it encodes.

2.2 LLM-Based Knowledge Graph Construction

LLMs have recently been applied to many steps of KG construction, from ontology engineering to knowledge extraction and fusion [6]. They can read and write RDF and SPARQL with reasonable accuracy [12, 25].

For the conversion of structured data, two main directions have emerged: direct triple extraction and mapping generation.

2.2.1 Direct Triple Extraction

In the first direction, the LLM receives the source data, usually in chunks or row by row, and outputs RDF triples directly. Tuozzo et al. [32] does a small qualitative feasibility study to evaluate whether a GPT model can be used to convert a CSV file of KG quality metrics into RDF triples that conforms to the Data Quality Vocabulary.

Pellegrino and Tuozzo [26] extend this study to investigate the effectiveness of different prompting strategies with various LLMs. Due to input limitations, they restrict their experiments to 10 rows of the CSV file. They report that the models struggle with incomplete outputs, instability across runs, and hallucinations as the number of columns grows.

Hariri et al. [14] convert archaeological CSV data into RDF that conforms to CIDOC CRM. They split the data into chunks of 20 rows and compare the performance of various prompting strategies across three settings: no ontology, the full ontology, and a subset curated by a domain expert. The setting without an ontology performs worst, and the curated subset performs best.

Direct triple extraction has two inherent limitations. First, its cost grows with the size of the data, since the LLM is invoked for every chunk. Second, the LLM makes its modeling decisions anew for every chunk, so the same entity may receive different IRIs or types in different chunks. Moreover, no reusable artifact is produced that could be inspected, corrected, or executed again on new data.

It is also to be noted that in these approaches, the target ontology was given, so the task is ontology population, and the LLM is not deciding the semantic model from scratch. If an LLM agent has to make modelling decisions such as which tables become edges or which predicates to use, that is a harder task than the one these studies evaluate.

2.2.2 Mapping Generation

In the second direction, the LLM generates a declarative mapping, which a mapping engine then executes.

Hofer et al. [16] let LLMs generate RML mappings from a single JSON record of IMDB data and a hand-selected subset of the DBpedia ontology, to which they deliberately added confusable properties. Mappings with syntax errors are given back to the model together with the parser's error message for up to two repair rounds. Valid mappings are executed with an RML engine, and the resulting KG is compared with a manually created reference. Of five commercial models, only Claude 3 Opus and GPT-4 regularly produced mappings that yielded triples.

OntoGenix [33] goes one step further and also designs the ontology. Its agents first derive an ontology structure from a statistical summary of a CSV file, then write the ontology in Turtle, and finally generate an RML mapping that is executed with Morph-KGC. It experiments on 6 kaggle datasets but each dataset is only a single-csv file. Compared with ontologies built by humans for the same six datasets, the resulting ontologies are richer in annotations but structurally shallower, and they contain more modeling pitfalls.

Schmidt et al. [30] generate YARRRML and RML mappings for three manufacturing

data sources at Bosch, exported from relational databases as CSV files, against small existing ontologies. They vary how much data the model sees (column headers only, or the first 20 or 200 rows), how the ontology is reduced before prompting, and whether the prompt contains a mapping template or retrieved examples. Compared with expert-written mappings, the best configurations reach a relaxed triple F1 of about 0.7, while exact matching drops sharply because the generated IRIs differ from the reference.

Trajanoska et al. [31] map relational data to schema.org with a multi-agent system that processes one table at a time. A mapping agent assigns schema.org terms to columns, using candidate terms retrieved from an index over the vocabulary. A relation agent identifies primary and foreign keys and creates links between entities, and a validator agent reviews both steps. The output is an assignment of terms rather than an executable mapping, and its accuracy was judged manually on databases from five domains of the Spider benchmark.

With the exception of OntoGenix, these approaches assume that a target ontology is given, and the LLM only selects terms from it. Moreover, they are evaluated on small datasets: a single JSON record, single CSV files, or a fixed set of test cases.

2.3 Competency-Question-Driven Modeling

Several approaches use competency questions to guide LLM-based ontology engineering. CQs were introduced by Grüninger and Fox [13] as natural-language questions that an ontology must be able to answer.

The central idea is that the design and evaluation of an ontology are the same activity: CQs define the requirements up front and then serve as the test that the axiomatised ontology must pass. Later methodologies kept CQs as their requirements mechanism but changed how they are checked. Instead of proving logical entailment, as Grüninger and Fox did, they test CQs as queries against an implemented ontology, typically in SPARQL. This later, query-based sense is the one used in this thesis.

With LLMs, CQs have also become an input to ontology generation, translating natural-language questions into formal ontologies. Ontogenia [20] lets an LLM output an OWL ontology using 14 CQs about African wildlifeas input. There is no data, the CQs are the only description of the domain. The model first interprets each CQ, then splits it into subject, predicate, and object to identify classes and properties, extends the ontology with restrictions, and finally justifies its decisions and tests the ontology on example instances.

Saeedizade and Blomqvist [29] generate OWL ontologies from ontology stories and their CQs. They point out that such input describes only the desired outcome, not how to model it. They compare zero-shot prompting, chain-of-thought variants, and two decomposition strategies across a range of models, and find that only GPT-3.5 and GPT-4 produce usable OWL. Their CQbyCQ technique processes one CQ at a time and merges each result into the growing ontology.

Evaluation is CQ-based: for each CQ, they check whether a SPARQL query answering it can be written against the generated ontology. This is particularly relevant for the purpose of evaluation in our experiments as well, as seen in section todo. Ignoring minor errors, GPT-4 with CQbyCQ comes close to the submissions of their student groups. It does particularly well on object properties and reified relations, but less well on restrictions. A follow-up study [21] develops both techniques further and reports that o1-preview with Ontogenia outperforms novice ontology engineers.

Taken together, these works suggest that CQs are a promising tool for our setting, in both of the roles that Grüninger and Fox [13] intended. As an input, CQs have proven to be a workable basis for LLM-based modeling and even as an evaluation criterion, CQs give a direct and interpretable measure of whether a model serves its purpose, whether checked by writing a query for each question [29], by scoring extracted triples against domain questions [14], or by executing paired queries over the source database and the generated graph [36].

Since CQs describe what a graph should be able to answer rather than how it should be modeled, they apply to the conversion of existing data just as they apply to designing an ontology from scratch. This makes them a natural fit for the modeling decisions that the source schema leaves open. So far, however, CQs have been given to LLMs as input only when designing ontologies, and the studies always included them, so their effect on the modeling has not been isolated. In our thesis, we aim to extend this line of work to the conversion of structured data and test directly whether supplying CQs changes the LLM's modeling decisions.

2.4 Evaluation in Related Work

Related works evaluate LLM-based KG construction at several levels. At the syntactic level, the output is parsed, for example with rdflib. Martin et al. [22] sort outputs into valid, recoverable, invalid, and empty.

Hofer et al. [16] additionally give the model up to two rounds to repair invalid outputs. At the level of the graph, the output is compared with a manually created reference. This comparison uses precision and recall of triples [14, 22] or the F1 score of exact graph matches [16].

Pellegrino and Tuozzo [26] count the correctly identified entities, dimensions, and metrics, and repeat every experiment five times to measure stability. BLINKG evaluates individual mapping decisions instead of the resulting graph [7].

When the LLM also designs the ontology, its quality is assessed with structural metrics such as OQuaRE and with the pitfall scanner OOPS! [33, 18]. Finally, Kommineni et al. [18] and Hariri et al. [14] test whether the generated KG can answer a set of competency questions.

Comparisons with a reference KG presuppose a fixed target vocabulary. Since we do not give a target ontology, a semantic KG produced by our approach may use different, but equally valid, classes and predicates than any reference. We therefore evaluate against the criteria defined in Section 1.2. Where possible, we use objective measures for fidelity, such as coverage and soundness. For the quality of the semantic modeling, we use a qualitative assessment, and for usefulness, we test whether competency questions can be answered.

3 Background

This chapter introduces the relevant concepts that our approach uses or builds upon.

3.1 SPARQL CONSTRUCT Queries

The central artifact of this thesis is a set of SPARQL CONSTRUCT queries: our approach generates these queries, and executing them on the input generic KG yields the semantic KG. The quality of the output therefore depends entirely on the quality of the queries. Before describing how our approach generates such queries, we explain how a CONSTRUCT query is structured.

SPARQL [15] supports four query forms, one of which is the CONSTRUCT form. It returns an RDF graph constructed by substituting variables in a set of triple templates.

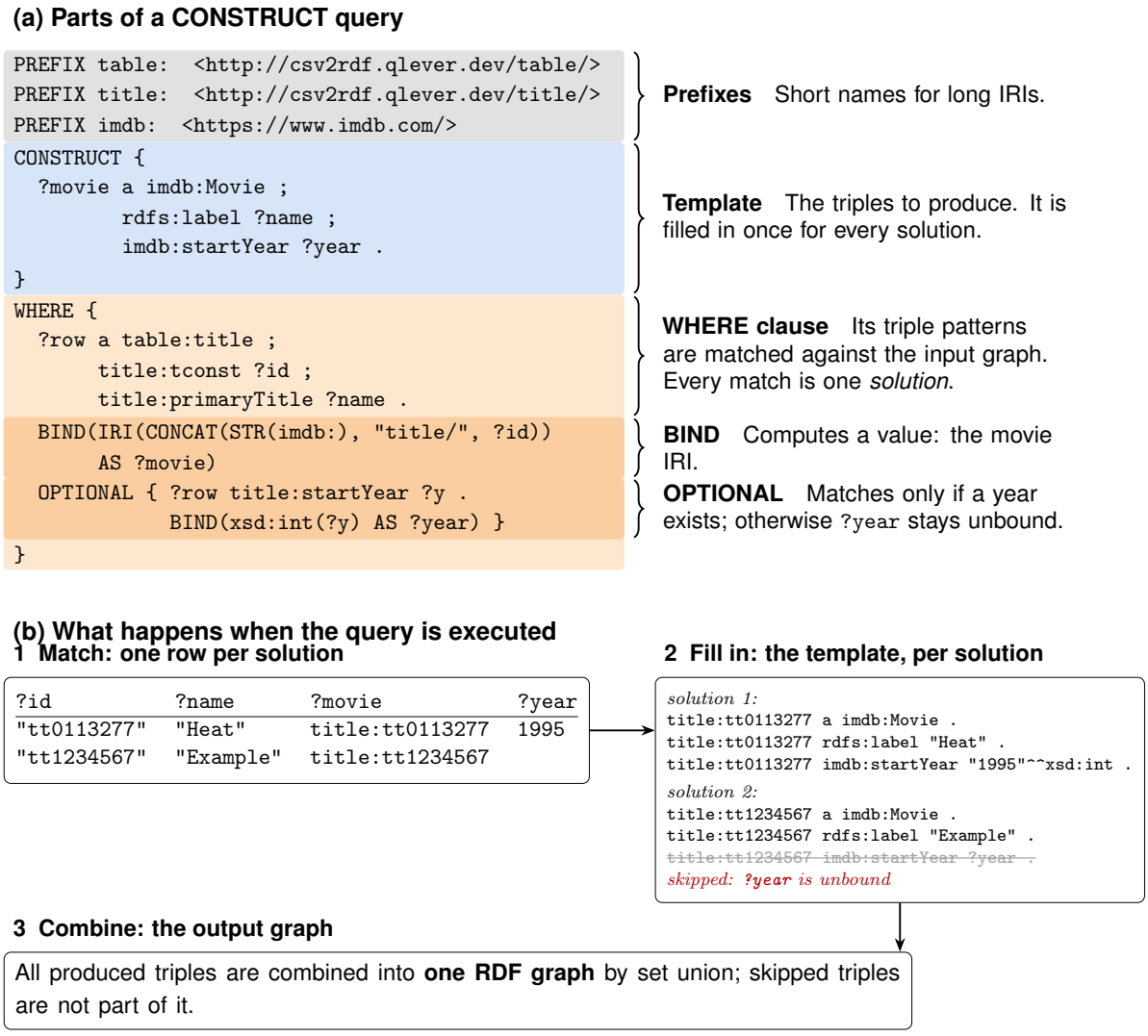


Figure 4: A CONSTRUCT query over a sample RDF KG

As shown in Figure 4, a CONSTRUCT query consists of two parts:

- a *template* which describes the triples that the query produces

- a WHERE clause which describes which data the query reads from the input graph. Its triple patterns are matched against the input graph.

For every solution of the WHERE clause, the variables in the template are replaced by their values, and the resulting triples are added to the output graph. The output graph is the set union of these triples.

So, in the example given in Figure 4, the template states that a movie has the type `imdb:Movie`, a label, and a start year. In the WHERE clause, the first pattern selects every node of class `table:title`, that is, every row of the table `title` in the generic KG. The other two patterns read the identifier of each such row into `?id` and its title name into `?name`. All three patterns must match for a row to be selected.

BIND computes a new value from values that were already matched, and assigns it to a new variable. In the figure, BIND appends the identifier of the movie to the IMDb namespace and converts the resulting string into an IRI with the function `IRI`. This is how a query mints IRIs for the entities it creates.

OPTIONAL marks a part of the WHERE clause that may fail to match without discarding the entire row. In the figure, the start year is optional. If a row has a start year, `?y` is bound to it, and the inner BIND converts it into an integer `?year`. If a row has no start year, the row is still selected, but `?y` and `?year` remain *unbound*, that is, they have no value. In this case, the template triple that uses `?year` is skipped for this row, and the output graph contains no triple about the start year of this movie.

Execution: Figure 4b shows how a SPARQL engine executes the query on two rows of the input graph. Execution proceeds in three steps [15].

1. *Matching.* The engine finds all ways in which the WHERE clause matches the input graph. Each match is called a *solution* and assigns a value to every variable that the match binds. In the figure, the two rows yield two solutions. The second solution leaves `?year` unbound, because its row has no start year.

2. *Instantiation.* For each solution, the engine replaces the variables in the template by their values. Each solution thus yields a copy of the template with concrete values. If a template triple contains a variable that is unbound in this solution,

that triple is left out, while the other triples of the same solution are kept. In the figure, this happens to the start-year triple of the second solution. A triple is also left out if it is not valid RDF, for example if a literal ends up in the subject or predicate position. Template triples without any variables are copied into the output unchanged.

3. *Combination.* The engine combines the triples of all solutions into one RDF graph by set union.

Caveats: While the execution of a CONSTRUCT query is straightforward, it has some caveats that are important for our approach.

- *Required patterns discard whole rows.* Every triple pattern of the WHERE clause that is not inside OPTIONAL must match [15]. If a row has no value for such a pattern, the row yields no solution, and none of its triples appear in the output. In the figure, a title without a primary title would therefore disappear entirely. The same happens if a predicate in the WHERE clause is misspelled: the pattern matches nothing, and every row is discarded. Values that may be missing must therefore be read in OPTIONAL.

- *IRIs must be minted consistently.* In RDF, two different IRIs are two different nodes. A query therefore refers to the same entity from several places only if it computes the same IRI each time. If two parts of a query mint the IRI of a person with different expressions, the output contains two separate nodes, and nothing in the graph indicates that they describe the same person.

- *Blank nodes are fresh per solution.* A blank node in the template denotes a new node for every solution [15]. Triples from different solutions therefore never share a blank node. Blank nodes are thus unsuitable for entities that are referenced from several solutions or several queries.

A CONSTRUCT query is executed by a SPARQL engine. In this thesis, we use QLever [5], which loads the generic KG G_D and materializes the output graph $Q(G_D)$. The size of the query depends only on the schema of D, not on the number of rows. The query is therefore a compact representation of the entire conversion, which can be read, corrected, and executed again on new data.

3.2 LLM Agents and Tool Calling

An LLM agent is an LLM that solves a task by interleaving reasoning with actions in an environment, and by observing the results of these actions [37]. In current LLM APIs, actions are implemented as *tool calls*. The agent is given a set of tools, each described by a name, a description, and a schema of its parameters. In each step, the LLM either produces a final answer or emits a call to one of the tools with required arguments. The runtime executes the call and returns its result to the LLM as a new message, and the LLM continues with the next step. This loop ends when the LLM produces its final answer or when a maximum number of steps is reached.

The capabilities of an agent are thus determined by its tools. A *general-purpose AI coding agent* is equipped with generic tools: reading and writing files, searching a code base, and running shell commands. With these tools, it can solve a wide range of tasks, for example by writing and executing scripts. OpenCode [1] is an open-source coding agent of this kind, which runs in the terminal and supports models from many providers. One can also configure their own LLM instances to work with the agent.

3.3 GRASP

GRASP [35] is an LLM-based agent originally designed for generating SPARQL queries over RDF knowledge graphs. Its central idea is that the agent is not given a description of the KG in advance, but explores it itself. Through a set of tools, the agent searches indices of the entities, properties, and literals of the KG, lists the triples of individual entities, and executes SPARQL queries. In this way, it learns how the KG is structured and what data it contains, before it writes its final query. In this thesis, we aim to extend GRASP with a new task, in which the output is a set of CONSTRUCT queries instead of a SELECT query. In the following, we describe the features of GRASP that this task particularly uses.

Tools: The agent explores the input KG with a set of tools. The functions `search_entity`, `search_property`, and `search_literal` search for specific data in the KG. For this, GRASP builds three search indices per KG: a fuzzy index for entities and an embedding index each for properties, and for literals. The function `search_property_of_entity` searches the properties used by a given entity, and `search_object_of_property` searches the common objects of a given property. The function `execute` runs an arbitrary SPARQL query and returns a preview of its results. If a shape index is available, the functions `search_shape` and `get_shape` additionally return the shapes of classes.

Shapes: A shape index describes the class structure of the input KG. Typically, for each class, the shape lists the properties that its instances use, together with the type of their values: a datatype for literals, or, for properties that point to other entities, the class of these entities. In addition, the shape lists the incoming properties of the class in a separate list (`incoming`): properties of other classes whose values are instances of this class, together with the class they come from.

```
{
  "iri": "<table/person>",
  "profile": {
    "total_entities": 4,
    "properties": [
      {
        "iri": "<person/id>",
        "triple_count": 5,
        "entity_count": 4,
        "targets": [{ "type": "literal", "datatype": "xsd:int", "triple_count": 4 }]
      },
      {
        "iri": "<person/name>",
        "triple_count": 4,
        "entity_count": 4,
        "targets": [{ "type": "literal", "datatype": "xsd:string","triple_count": 4 }]
      },
    "incoming": [
      {
        "iri": "<ref/movie/director_id>",
        "triple_count": 745,
        "entity_count": 3,
        "targets": [{ "type": "class", "iri": "<table/movie>","triple_count": 745 }]
      }]
  },
}
```

Figure 5: GRASP sample shape format for one class

Notes: Before the agent is used on a KG, GRASP also has the feature to let it explore the KG freely and record its observations as notes. These notes may then be used in other tasks so that that it starts with knowledge that would otherwise have to be gathered again with exploration. Figure 6 shows sample notes recorded for a database related to superheroes. They describe both schema and data characteristics.

1. Main class structure: `person` has `id`, `person_name`, `full_name` (optional), `height_cm`, `weight_kg`, and foreign key references to `colour(eye/hair/skin)` , `gender`, `race`, `publisher`, and `alignment` tables.
2. Naming patterns: Tables use `table/{name}`. Rows use `row/{table}/{id}` or `_rowN` for junction tables. Properties: `ref/{table}/{fk}_id` for foreign keys, `{table}/{field}` for direct properties.

Figure 6: GRASP sample structural notes

4 Approach

In Section 1.2, we defined our task: given the generic KG G_D of a structured dataset D, find a set of CONSTRUCT queries Q such that $Q(G_D)$ is a semantic KG of D. This chapter describes how we propose to implement this task.

Our approach follows the idea of GRASP (Section 3.3): instead of giving an LLM a description of the data, we let an LLM agent explore the data itself, and write its result as SPARQL queries. The difference to the original use of GRASP is the kind of query. The agent does not write a SELECT query that answers a question, but CONSTRUCT queries that remodel an entire KG.

Figure 7 gives an overview. The approach consists of four steps, each described in its own section:

1. **Convert** D into a generic KG G_D with a direct mapping (Section 4.1).

2. **Prepare** G_D for exploration by building the notes, search indices, and shapes of GRASP (Section 4.2).

3. **Generate** the CONSTRUCT queries with a generation agent, and check them with an independent review agent (Section 4.3).

4. **Materialize** the semantic KG by executing the queries on G_D.

Finally, Section 4.4 describes the baseline against which we compare our approach.

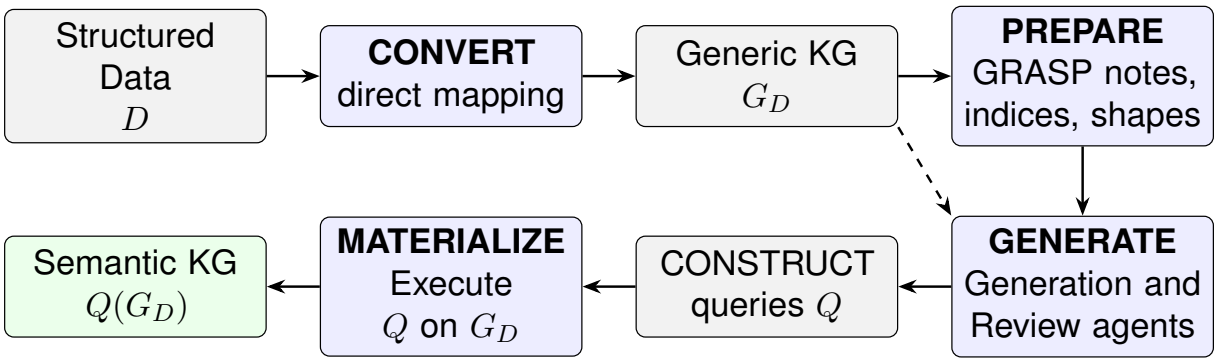


Figure 7: Overview of generic-to-semantic KG pipeline

4.1 Structured Data to Generic KG

Our task, as defined in Section 1.2, starts from an RDF KG that is to be modeled semantically, regardless of the format in which the data was originally stored. The first step therefore, is to convert the source data D into a generic KG G_D. Converting every source into a generic KG first keeps the rest of the pipeline independent of the source format: the agent explores the data with SPARQL, and writes its result as SPARQL CONSTRUCT queries over the same graph.

Mapping: In this thesis, we only consider relational data for which we refer the W3C Direct Mapping [2] to generate the generic KG.

- The row becomes a node whose IRI consists of a base IRI, the table name, and the value of the primary key. If the primary key is missing, the row is identified by its position, as `_row`N. A composite primary key is written as `column=value` pairs joined with `;`.

- The row receives an `rdf:type` triple with a class named after its table.

- Every non-null cell yields a triple with a predicate named after the table and the column.

- Where the source declares column types, we follow R2RML [9]: integer types map to `xsd:integer`, `NUMERIC` and `DECIMAL` to `xsd:decimal`, floating-point types to `xsd:double`, and boolean, date, and timestamp types to `xsd:boolean`, `xsd:date`, and `xsd:dateTime`. A value receives a datatype only if it also matches that datatype's lexical form; otherwise, and for all other cases, it is written as a plain string.

- For a declared foreign key, the row receives an edge to the referenced row and optionally may also emit a triple with the foreign key value as a literal. For composite primary keys, the foreign key value is emitted accordingly to that single node.

If a schema is known, the converter additionally emits, once per table, a class declaration and the `rdfs:domain` and `rdfs:range` of each predicate.

Table 1 summarizes the mapping rules and gives examples.

Source	Generic KG	Example
Row	`<.../row/{table}/{key}>`	`<row/title/tt0113277>`
Table	`rdf:type <.../table/{table}>`	`<table/title>`
Cell	`<.../{table}/{column}>`	`<title/primaryTitle>` `"Heat"`
Foreign key	`<.../ref/{table}/{column}>`	`<ref/principals/nconst>`
Typed column	XSD datatype, if the declared type maps to one and the value is valid	`"1995"^^xsd:integer`

Table 1: Rules of the direct mapping

Conversion Practices: Real-world databases deviate from the idealized picture of the Direct Mapping in several ways. The converter therefore applies the following practices to all datasets:

- **Sanitizing special characters:** Table names, column names, and key values become parts of IRIs, but may contain characters that IRIs do not allow. Every character outside the unreserved set of RFC 3986 (ASCII letters, digits, and `-` `.` `_` `~`) is percent-encoded as its UTF-8 bytes.

  `image/jpeg`, a primary key $\xrightarrow{\text{percent-encode}}$ row IRI `<row/image_type/image%2Fjpeg>`

- **Null handling:** Values are trimmed of leading and trailing whitespace, and a cell yields no triple if it is then NULL or empty. By default, the converter treats only real NULL values and empty strings as missing. In addition, it accepts an optional list of null markers, that is, strings that stand for a missing value in a particular dataset. Text files such as CSV or TSV files, for example, cannot represent NULL, and therefore often write missing values as `\N`. Values that only mean “unknown” within the domain, such as `-` or `N/A`, are not interpreted and remain ordinary values, since deciding their meaning is a modeling decision left to the agent unless the dataset owner explicitly declares them as null markers.

- **Multi-valued cells:** Delimited files often contain cells with multiple values either separated by a delimiter or in an array, for instance, a list of genres `Action,Crime,Drama`. Besides the field delimiter of a file, the converters therefore accept an optional sub-delimiter, together with the columns it applies to, or all columns if none are given. A cell in such a column is split into one literal per value.

- **Sampling:** An agent does not need to see every row of a table to understand its structure, and a large KG slows down exploration. The converters can therefore optionally produce a sample of the data instead of converting it in full.

  Sampling is controlled by three parameters: a size threshold, a sample size, and a limit for foreign-key repair. Tables below the threshold (by default 5,000 rows) are always converted completely. From larger tables, a fixed number of rows (by default 2,000) is drawn uniformly at random using reservoir sampling [34].

  For datasets where schema is available, sampling the tables independently, breaks references: a sampled row may refer to a row in another table that was not part of that table's sample. To fix this, we apply *foreign-key repair*. While sampling, the converter records every value that a foreign key refers to. It then adds the rows that hold these values to their tables. Each row is looked up by the column that the foreign key names(which may not always be the primary key). References from completely converted tables are also recorded, because they can point into sampled tables as well.

  To keep the sample small, repair is limited in two ways. First, it goes only one level deep, so the references of the added rows are not followed. Second, it adds at most a fixed number of distinct values per referenced column (by default 1200). References beyond these limits remain dangling in the sampled KG.

  Reservoir sampling reads the table in a single pass and needs memory proportional to the sample size, not to the size of the table. If a random seed is given, the sample is reproducible.Moreover, sampling does not limit the result. A CONSTRUCT query set generated on a sample can be executed on the generic KG of the complete data.

Beyond these general practices, some datasets required specific handling. We describe these cases together with the datasets in Section 5.1.

4.2 Preparing the Generic KG

A generic KG of a database with hundreds of tables is too large to understand by browsing its triples. Before the agent starts, we therefore let GRASP build the components described in Section 3.3. We load G_D into a QLever instance [5], which serves as the SPARQL endpoint for all further steps, and run the setup of GRASP once per dataset, as per the following:

1. **Info.** The setup task determines the prefixes of G_D and writes a short description of the input KG.

2. **Structural notes.** Over three rounds, the agent repeatedly picks a class or property in a part of G_D that is not yet well covered, explores its connections, and records notes on the structure and content of G_D, such as those in Figure 6.

3. **Search indices.** The setup task writes the SPARQL queries that select the entities, properties, and literals of G_D. GRASP executes them and builds the three search indices.

4. **Shapes.** Given the structural notes and search indices, the setup task determines how G_D assigns classes to instances and declares its schema. GRASP then discovers the classes, profiles their instances, and builds the shape index, with shapes such as the one in Figure 5. Since every row of the generic KG is typed with the class of its table, the setup reliably finds the class of every row, and each shape summarizes one table.

4.3 Generating CONSTRUCT Queries

The core of our approach is the generation of the CONSTRUCT queries. The manual process it replaces consists of understanding the data, designing a model, and writing and checking the mapping. We divide this work between two agents, both implemented as tasks of GRASP and equipped with its exploration tools:

- a *generation agent* (`construct-gen`), which explores G_D, designs the model, and writes the queries,

- and an independent *review agent* (`construct-review`), which checks the result against the data.

Figure 8 shows how the two agents interact. The generation agent writes and tests queries until it is satisfied, and then submits them as a set. Every accepted submission is checked by the review agent, which returns a verdict together with feedback. Unless the verdict states that the queries are done, the generation agent revises them and submits again, up to a configured maximum number of review rounds.

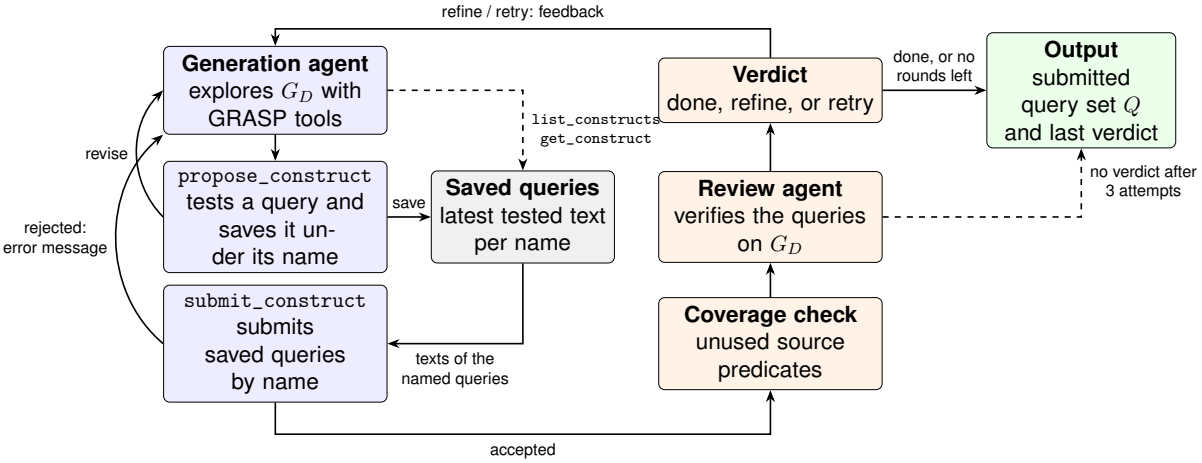


Figure 8: Generation and review of CONSTRUCT queries.

In the following, we first describe the generation agent and how it manages its queries, and then the review agent and the loop between the two.

Generation agent: The prompt of the generation agent describes the task: analyze the input generic KG, G_D and produce CONSTRUCT queries that model it as a well-formed, semantically correct KG. Since G_D may already model some of its data well, the agent should keep or refine such parts, and remodel the rest. The prompt asks the agent to proceed in the following steps, which mirror the manual process:

1. **Explore** G_D, starting from the notes and shapes, and run targeted queries, if necessary, to fill gaps in its understanding of the source data.

2. **Design** the mapping. Decide which entities to model, which classes to assign, and which fields are relationships between entities. Mint IRIs from stable identifiers, in a namespace that reflects the domain. Determine how missing values look in the data, and how to handle fields where units, currencies, or scales may be attached. Finally, decide how many queries are needed to cover all the data without loss.

3. **Test** each query under a short, descriptive name, and check that it runs and produces well-formed output.

4. Optionally, **check** the design against the competency questions, if given, by sketching a SPARQL query for each question over the graph the queries would produce. A question that cannot be answered, or only with an overly complex query, points to a gap in the model.

5. **Submit** the queries once they are complete.

The competency questions are presented as a minimum, not as the scope of the model: the agent must not narrow the model to the questions at the expense of coverage.

Rules: In initial experiments, we observed several recurring errors in the generated queries. We therefore added the following rules to the prompt to steer the agent away from them:

- Every entity should receive an `rdfs:label`, but the label must not be a required pattern, since a missing label would otherwise drop the entity.

- Values that are actually missing should be read in OPTIONAL, so that they do not suppress the other triples of the entity. OPTIONAL however, should not be used as a blanket pattern for values that are always present.

- An entity referenced by several queries must be minted with the same IRI pattern in each, and the same predicates and namespace be used for the same concept, so that the queries combine into one coherent graph.

- A unit, currency, or scale encoded in a column name, as in `height_cm`, should ideally not be lost while conversion.

Generation Agent Tools: In an early version of our approach, the agent submitted its queries by writing out their full text. This caused two problems. First, the agent had to retype queries it had already tested, and could introduce new errors in doing so. Second, when resubmitting after review feedback, it sometimes left out queries it had not changed, and their data silently disappeared from the output.

We therefore separate testing a query from submitting it. The generation agent works with two collections: the *saved queries*, which hold the latest tested text of every query under its name, and the *submitted set*, which is the final set of queries that are submitted

for review. Four tools operate on these collections:

- `propose_construct` takes a name and a query, saves the query under the name, and tests it on G_D. It returns up to 20 of the produced triples, or the error message.

- `list_constructs` lists all saved queries, with whether each is submitted and whether its latest test succeeded.

- `get_construct` takes a name and returns the exact saved text of the query.

- `submit_construct` takes the names of all queries that form the set, and replaces the submitted set by these saved queries. It reports which queries were added, updated, and dropped.

A query goes through the following life cycle:

- **Propose.** The agent proposes a new CONSTRUCT query under a name and saves it. The agent can propose a query with the same name to update it, and the saved text is always the latest version.

- **Test.** The agent tests the proposed query with `propose_construct`. A test succeeds if the query runs and produces at least one triple. If the query fails or produces no triples, the agent is asked to revise the query. Since the saved text will later be submitted, a `LIMIT` or `OFFSET` that the agent adds itself would truncate the final output. It is therefore removed from the saved text, and the agent is informed.

- **Inspect.** The agent can check which queries it has saved and submitted, and read the exact text of a saved query, for example to reuse its IRI patterns in another query.

- **Submit.** The agent submits by calling `submit_construct` with the names of all queries that form the set. From the saved queries, it creates the submitted set as per these names. If a name is empty, duplicated, has no saved query, or belongs to a query whose latest test failed, the call is rejected, and the submitted set remains unchanged. An accepted submission starts the review.

- **Revise.** After review feedback, the agent tests revised versions of its queries under

their existing names, and submits the set again. Queries that it did not revise keep their saved text, and are submitted unchanged.

- **Delete.** A previously submitted query whose name is left out of a new submission is dropped from the set, for example when its data is now produced by another query. The agent is asked to state the reason in the argument `removed`: why the query is no longer needed, and where its data is covered now. The reasons are collected across all review rounds.

This life cycle resolves both problems of the earlier version. The submitted query is always the one that passed the test, since the agent never retypes a query, and no query disappears unnoticed: every submission reports the dropped queries, and dropping a query requires a deliberate decision, ideally with a stated reason.

Review agent: Iterative refinement, in which an LLM revises its output based on feedback, is an established technique for improving LLM output. However, when an LLM only reconsiders its own output, such self-correction often fails to improve the result. The submitted queries are therefore checked by an independent review agent. It has the same exploration tools and access to G_D as the generation agent, but not its trace, so it does not inherit the generation agent's assumptions. The reviewer is instructed not to trust the queries as written, but to verify every aspect by querying G_D, and to report only issues that it has verified.

Every accepted submission starts a review round, which proceeds in three stages. First, the coverage of the source data is computed mechanically. Then, the review agent checks the submitted queries against a checklist, with the computed coverage as additional input. Finally, the reviewer returns a verdict, which decides how the run continues. We describe the three stages in this order.

Computing source coverage. Of all aspects that the reviewer has to check, source coverage is the most important. As per our initial experiments, it was clear that as the schema size increases, it becomes more likely that the agent misses a lot of the columns from the source data, in its proposed queries. Before the reviewer starts, we therefore compute coverage mechanically by comparing the predicates of G_D with the predicates that the submitted queries mention.

1. **Listing the source predicates.** A SPARQL query lists every predicate of G_D, together with its number of triples. Since every column of a table becomes a

predicate in the generic KG, this list corresponds to the columns of D.

2. **Collecting the referenced IRIs.** For every submitted query, all IRIs that the query mentions are collected. These are full IRIs and prefixed names, which are expanded using the prefix declarations of the query. The keyword `a`, the SPARQL shorthand for `rdf:type`, counts as a reference to `rdf:type`.

3. **Comparing.** A source predicate counts as covered if at least one query mentions it.

The result is added to the reviewer's prompt: the number and percentage of covered predicates, and the list of uncovered predicates, ordered by their number of triples, so that the predicates that carry the most data come first. The check is an upper bound on the actual coverage, since a predicate that a query mentions may still lose values, for example through a filter or a missing OPTIONAL. But it is intended to guide the review agent's attention so that it can focus on the most important issues first. If the source predicates cannot be listed, for example because the endpoint fails, the reviewer is told to check coverage itself.

Checklist. The review agent receives the submitted queries, the computed coverage, and a checklist of aspects to verify:

- **Source coverage.** All data of G_D is accounted for by the queries, and nothing significant is missing. For every uncovered predicate in the computed coverage, the reviewer reports whether it matters and what should be added to cover it, unless its data is represented in some other way.

- **Value types.** For each predicate in the output, the type of its values is appropriate: a literal where a value is expected, and an IRI where a relationship to another entity is intended.

- **Relationship modeling.** The reviewer should first query G_D to independently identify which relationships exist between entities in the data, and then checks that the queries model them. Every field that refers to another entity must become an IRI link to that entity, not remain a raw literal, regardless of whether the entity is minted in the same query or in a different one.

- **Consistency.** The same real-world entity is minted with the same IRI pattern wherever

it is referenced, within a query and across queries.

- **Namespace consistency.** All custom predicates and entity IRIs use one namespace, which reflects the domain of the data. The reviewer judges this from the full IRIs, not from the prefix names used in the queries.

Depending on the submission, the checklist contains further items. If competency questions are given, two items are added:

- **Competency question coverage.** For each question, the reviewer writes the SPARQL query that would answer it over the graph that the queries produce. It cannot execute this query, since that graph does not exist yet, but writing it serves as a usability test of the model: a question that cannot be answered, or only with an overly complex query, points to a gap in the model, and the reviewer names the part of the model responsible.

- **Competency question scope.** It is specified to the reviewer that the questions are a minimum, not the full scope of the model. The reviewer checks that the model was not narrowed to the questions at the expense of source coverage.

If several queries are submitted, one more item is added:

- **Cross-query consistency.** The queries form one coherent graph: the same entity is named and modeled consistently across queries, the same predicates and namespace are used for the same relationship, and every entity type referenced by more than one query is minted with the same IRI pattern.

Verdict and review loop. The reviewer ends its round with one of three verdicts, together with feedback that describes the issues it found: *done*, if the queries are correct and complete; *refine*, if they need targeted fixes; or *retry*, if the overall approach must be reworked. For *refine* and *retry*, the feedback is passed to the generation agent as a new message, and the agent revises and resubmits its queries as described above, which starts the next round. The loop ends when the verdict is *done*, or when the configured maximum number of review rounds is reached. In the latter case, the output is marked as submitted with unresolved issues, so that such runs can be distinguished in the evaluation.

The reviewer is itself an LLM agent and can fail: its run may end with an error, or without a verdict. Such a failure says nothing about the queries, so the reviewer is run again on the same queries, up to three times. These attempts do not count as review rounds. Only if all attempts fail are the queries output without a review, marked as unreviewed.

Output. A run produces the submitted set of named CONSTRUCT queries, the reasons given for dropped queries, and the last verdict, with its feedback, the number of review rounds, and whether the review budget was exhausted. We also store the complete traces of both agents, which we use to analyze the behavior of the agents in Chapter 5.

4.4 Baseline

We compare our approach with a baseline that uses a general-purpose coding agent for the same task. We use OpenCode [1], an open-source agent that runs in the terminal and works on the files of a project directory (Section 3.2). Its tools are generic: it can read and write files, search them, and run shell commands.

Task and input: The baseline solves the same task as our generation agent: it writes a set of CONSTRUCT queries that transform the generic KG G_D into a semantic KG. Unlike our approach, it does not access G_D through a SPARQL endpoint, but receives it as an N-Triples file, the same sample that our approach explores via a SPARQL endpoint. In the corresponding configuration, it additionally receives the competency questions. It writes its queries as files into a given output folder, and refrains from accessing any other files.

Since it's a general-purpose coding agent, the baseline has to explore the data with its generic tools, for example by searching the N-Triples file for the predicates of a table. For the same reason, it cannot directly execute the CONSTRUCT queries it writes. Whether and how it tests them is left to the agent so as to not constrain its behavior.

Prompt: To make the comparison fair, the prompt of the baseline mirrors that of our generation agent wherever possible. It describes the same task, asks the agent to proceed in the same steps, and contains the same rules (Section 4.3). Since the baseline has no independent reviewer, its prompt additionally contains the reviewer's checklist as a self-review: before finishing, the agent is asked to verify its queries against the input file, as per the given checklist.

Differences to our approach: The baseline thus receives the same data, task, and instructions, and produces the same kind of output. It differs in three ways:

- **Exploration.** Unlike our approach that can leverage the exploration tools, notes, search indices, shapes provided by GRASP, the baseline explores the data only as a file, with generic tools.

- **Testing.** The baseline is free to test its queries for example using rdflib or Oxigraph to run them on the N-Triples file, but it does not have the same testing as our approach, where the agent can test each query on the SPARQL endpoint and receive a sample of the produced triples or an error message and revise the query until it runs successfully.

- **Review.** It is upto the baseline agent to decide how to generate and test the queries. Its queries are not checked by an independent reviewer with a mechanical coverage check, but only by its own self-review.

Since both approaches produce CONSTRUCT queries over the same generic KG, we materialize and evaluate their outputs in the same way. The CONSTRUCT queries are executed on the QLever endpoint of G_D, and the resulting triples are written to an N-Triples file.

5 Evaluation

In this chapter, we evaluate our proposed approach against the baseline. We first describe the datasets, then the evaluation setup, and finally report the results.

5.1 Datasets

We evaluate on eight relational datasets in total. The primary evaluation is done using three public datasets: IMDb, Mondial, and MusicBrainz. They were chosen to cover a range of domains and of schema sizes, from 7 tables to 239. We also include five datasets from the BIRD benchmark [19] in order to perform an ablation study on the effect of competency questions on the modeling. The BIRD datasets are all SQLite databases, and they are modeled with and without competency questions. Table 2 gives an overview of the datasets, their formats, and their sizes. The number of competency questions (CQs) is given only for the BIRD datasets, which are the only ones that come with such questions.

Dataset	Format	Tables	Columns	Rows	FKs	CQs
IMDb	TSV	7	39	207.4 M	7‡	–
Mondial	SQLite†	47	192	56,702	74	–
MusicBrainz	TSV + DDL	239	1,588	25.1 M	473	–
superhero	SQLite	10	31	10,614	11	16
california_schools	SQLite	3	89	29,941	2	9
formula_1	SQLite	13	94	493,257	19	20
card_games	SQLite	6	115	803,445	4	16
european_football_2	SQLite	7	199	222,796	31	16

Table 2: Overview of the datasets. *FKs* counts declared foreign keys. *CQs* gives the number of competency questions given to the agents, out of all questions of BIRD mini-dev for that database. † converted from PostgreSQL. ‡ declared by us using the schema documentation.

5.1.1 IMDb

The IMDb non-commercial datasets [17] describe movies, series, episodes, and the people involved in them. They consist of seven tab-separated files, one per table: `title.basics`, `title.akas` (alternative titles per region and language), `title.crew`, `title.episode`, `title.principals` (the credits of a person on a title), `title.ratings`, and `name.basics`. While the current export from IMDb contains 207.4 million rows, its schema is relatively simpler. In converting the TSV files into a generic KG, we take note of three specific cases:

- **No schema.** The files contain only a header row, without types, primary keys, or foreign keys. Without a schema, every row would be identified by its position and the

graph would contain no edges between rows. We therefore refer the documentation provided to declare the primary keys (e.g. `tconst` for `title.basics`, and the composite key `(titleId, ordering)` for `title.akas`) and seven foreign keys inline when calling the converter.

- **Null marker.** Since TSV cannot represent NULL, IMDb writes missing values as `\N`, which we declare as a null marker.

- **Multi-valued cells.** Columns such as `genres`, `primaryProfession`, and `knownForTitles` hold comma-separated lists, which are split into one literal per value with the sub-delimiter option in our converter. Lists of identifiers (`knownForTitles`, `directors`, `writers`) are split like any other column, but not linked to the rows they name. The columns `types` and `attributes` of `title.akas` separate their values with the control character `0x02` instead, which we did not split.

The input generic KG we use in our experiments is produced from the IMDb dump using our converter with a sample size of 100, sample threshold of 200 and an FK cap of 5000. Any table with at least 200 rows is reduced to a uniform random sample of 100 rows. Sampling tables independently leaves foreign keys pointing at rows that were not selected. The converter therefore performs one-hop FK repair: it adds back every parent row referenced by a sampled row, up to a cap of 5000 distinct values per referenced column. The result contains 10,421 triples having 700 titles, 200 people, and 100 rows in each of the five remaining tables.

On this sample, the analytical questions of Section 5.3.3 all have empty answers: no title of a sampled credit has a rating, no director has a person row, and the highest number of votes is just 5,368. Uniform per-table sampling cannot produce connected data, since foreign-key repair only adds the parents of sampled rows, not their children. For the evaluation, we therefore built a second, connected generic KG from the complete data, G_D^{pop}, in exactly the layout of G_D: all 3,092 titles with at least 100,000 votes, with their ratings, crew, 66,452 credits, 173,143 alternative titles, and 29,701 episodes, and all 32,198 credited persons. It has 2,505,248 triples and no dangling references. We execute the output CONSTRUCT queries on this graph and use the resulting semantic KG for the evaluation.

5.1.2 Mondial

Mondial [23] is a geographical database compiled from the CIA World Factbook and other sources. It describes countries, provinces, cities, organizations, and geographical

features such as rivers, lakes, mountains, islands, and deserts, and the relations between them, for example which river flows through which province, or which country is a member of which organization. It consists of 47 tables with 193 columns. It's schema has 74 foreign keys, and 30 tables with a composite primary key (a province, for example, is identified by its name together with its country). This makes it the most densely connected relational schema in our evaluation.

Mondial is distributed as PostgreSQL scripts. We convert it into SQLite first and then use our converter script. There are four specific cases that the converter handles separately for this dataset:

- **Composite column type.** Coordinates use a PostgreSQL composite type, `GeoCoord(Latitude, Longitude)`, whose values are written as tuples such as `(65,-18)`. Since SQLite has no composite types, each such column is replaced by two numeric columns, `<Column>_Latitude` and `<Column>_Longitude`, and every tuple is split into these two values.

- **Missing foreign keys.** The DDL declares no foreign keys at all. Mondial documents its referential dependencies in a separate PDF. We transcribed them into the converter, which adds them as foreign-key constraints and verifies the finished database with `PRAGMA foreign_key_check`.

- **Encoding errors.** Parts of the data are UTF-8 text that was once decoded as Windows-1252 and re-encoded, such as `ÃŽle-de-France` for `Île-de-France`. These are repaired before loading.

The input generic KG we use in our experiments is produced from the Mondial dump using our converter, with a sample size of 50, sample threshold of 100 and an FK cap of 1600. Tables with fewer than 100 rows are kept in full (9 tables, mostly small lookup tables such as `Continent` and `Sea`). Each of the remaining 38 tables is reduced to a uniform random sample of 50 rows. One-hop FK repair then adds back every parent row a sampled row references, up to 1,600 distinct values per referenced column. The central entity tables are still much larger than the sample size: 430 cities, 529 provinces, 228 countries. The result has 4,043 rows and 26,629 triples. Like IMDb, Mondial is modeled without CQs as well.

5.1.3 MusicBrainz

MusicBrainz [1] is an open encyclopedia of music metadata: artists, releases, recordings, works, labels, places, and events, as well as the edit history of its community of editors. We use the official sample dump [24], restricted to 239 tables, which is the largest schema in our evaluation: 1,588 columns and 473 foreign-key columns.

The dump consists of tab-separated PostgreSQL `COPY` files without a header; the schema is shipped separately as SQL DDL. We first add a header row to each file from the DDL, and then convert the files to generic KG, where our converter reads the schema directly from the DDL with the following special cases handled:

- **Keys and references in DDL.** MusicBrainz declares its keys and references not as SQL constraints, but in column comments (`-- PK`, `-- references artist.id`), so the converter reads them from there.

- **Composite primary keys.** Every column marked as `PK` belongs to the key, not only the first one. 67 tables have a composite key, for example `(recording, tag)` for `recording_tag`. Keeping only the first column would merge all rows that share it into one node so we infer the key from all columns marked as `PK`.

- **Implicit primary keys.** MusicBrainz ships no separate primary-key definitions, and 114 of the 239 tables declare their key only through a `SERIAL` column. We infer the key from that column type; without this, every reference to these tables would be dangling. Tables with neither marker get no key.

- **Schema-qualified names.** The Cover Art Archive tables are dumped as `cover_art_archive.art_type`, but referenced as `art_type`. The schema qualifier is removed, otherwise these files would receive no schema and all 112,142 references into them would be lost.

- **References to absent tables.** The DDL declares tables that the dump does not contain. References to them are dropped.

The input generic KG we use in our experiments is produced from the MusicBrainz dump using our converter, with a sample size of 5000, sample threshold of 30,000 and FK cap of 10000000. Because MusicBrainz is far larger than the other databases, we

[1] `https://musicbrainz.org/doc/MusicBrainz_Database`

use a much higher threshold and sample size. Tables with fewer than 30,000 rows are kept in full: 180 tables, including core entities such as `release`, `release_group`, `label` and `area`, and all type and lookup tables. Each of the remaining 59 tables is reduced to 5000 random rows. One-hop FK repair then adds back every referenced parent row. We set the cap to 10^7 values per referenced column, so it never limits repair. The result has 1.36M rows and 11.86M triples. Like IMDb and Mondial, MusicBrainz is modeled without CQs as well.

5.1.4 BIRD-SQL Databases

BIRD [19] is a text-to-SQL benchmark of 95 SQLite databases from 37 professional domains. Each database comes with natural-language questions, each paired with a gold SQL query, a hint (called *evidence*) that explains the domain knowledge the question needs, and a difficulty rating of *simple*, *moderate*, or *challenging*. We use its curated subset mini-dev [2], which contains 11 databases, and from these we select five of varying schema complexity: superhero, card_games, formula_1, european_football_2, and california_schools.

- **superhero** describes 750 superheroes with their publisher, race, gender, alignment, eye, hair, and skin colors, and two many-to-many tables that link heroes to superpowers and to attribute values. It is the smallest database, and the only one in which two tables (`hero_power`, `hero_attribute`) have no primary key, so their rows are identified by position (`_row`N).

- **card_games** describes the cards of the trading card game *Magic: The Gathering*. The table `cards` has 74 columns, and several of them hold lists in a single string, such as the sets in which a card was printed. All four foreign keys reference a non-key column (`cards.uuid`, `sets.code`).

- **formula_1** describes Formula 1 seasons, races, circuits, drivers, and constructors, with results, qualifying, lap times, pit stops, and championship standings. It has the most tables among the BIRD databases (13), the most foreign keys (19), and two tables with a composite primary key.

- **european_football_2** describes European football leagues, teams, players, and matches, with yearly attribute snapshots of players and teams. Its table `Match` has 115 columns: 22 lineup slots (`home_player_1` to `away_player_11`) with their pitch coordinates, betting odds from ten bookmakers, and eight columns (e.g. `goal`, `card`,

[2] `https://github.com/bird-bench/mini_dev`

`corner`) whose values are XML documents listing the events of the match. 28 of its 31 foreign keys reference unique columns such as `player_api_id` instead of the primary key.

- **california_schools** describes Californian schools, with their free and reduced-price meal statistics and SAT results. It has only three tables, but they are wide (49, 29, and 11 columns), with column names such as `Percent (%) Eligible Free (K-12)`. Some rows of `schools` and `satscores` describe a district, not a school.

Competency questions: The questions provided by BIRD-SQL for each database are written by domain annotators, they ask what a user of the data would want to know, and, since each comes with a gold SQL query, they have a known answer on D. Since only the BIRD databases come with such questions, they are the datasets used in our ablation study: each is modeled with and without competency questions, by both our approach and the baseline.

For each of the five databases, we split its mini-dev questions into a *guidance set* of about 30%, stratified by difficulty, and a *held-out set* of the rest. Only the guidance set is given to the agents as competency questions (Table 2, column *CQs*). We give the agents the question text only, without the evidence and the SQL query. The held-out questions are reserved for checking whether the produced KGs can answer questions that did not influence the modeling.

The BIRD databases are SQLite files, which our converter converts directly. Primary keys, column types, and foreign keys are read from SQLite's `PRAGMA` metadata, so no further schema is needed. Beyond the general practices of Section 4.1, there is one edge case in the BIRD databases that the converter handles:

- **Foreign keys to non-key columns.** The edge of a foreign key points to the IRI of the referenced row, and that IRI is minted from the primary key. When a foreign key references another unique column, as in `card_games` and `european_football_2`, the converter first builds a lookup from the referenced value to the primary key of its row, and uses it to resolve each reference. If a value names more than one row, no edge is emitted and the literal is kept. For the same reason, foreign-key repair during sampling looks rows up by the referenced column, not by the primary key; with a primary-key lookup, it would repair almost no references in the `european_football_2` dataset.

5.2 Setup

We evaluate on two groups of databases. The first group consists of the three datasets of increasing scale and schema complexity: IMDb, Mondial and MusicBrainz. For each, we convert a sample to a generic KG G_D as described above and run GRASP and the OpenCode baseline on it as they are, to compare how the two approaches remodel the underlying schemas. This is the main evaluation of our approach, and it is done without competency questions given to the agents.

The second group consists of five databases from the BIRD text-to-SQL benchmark as mentioned above, which we use for an ablation study on competency questions (CQs). Each system is run on each database twice, once without and once with a set of CQs supplied as modelling requirements. This is to evaluate the effect of CQs on the modeling quality of the resulting semantic knowledge graphs.

We evaluate the output of every run on three levels, which correspond to the criteria of Section 1.2: whether the queries execute at all, how well they model the data, and how usable the result is.

Syntactic validity: The first level is whether the output CONSTRUCT queries Q can be executed on the input KG G_D and materialise a valid graph. We send each CONSTRUCT query of a run to the QLever endpoint of its generic KG, receive N-Triples, and write the union of the results into one file, which is the output KG G. We count a run as valid if every query executes and the result parses as N-Triples. We record which queries needed intervention before they could be materialized, and of what kind.

LLM-as-a-judge: The second level is the quality of the semantic modeling and the fidelity of G. An exhaustive manual evaluation of every output, across all datasets, configurations, and runs, would require a considerable amount of manual effort. Instead, we let an LLM agent act as a judge to do an independent evaluation and score the output KGs. Individually, this score is only a rough estimate. Across all outputs, however, the scores indicate which approach generally performs better, and which kinds of issues are most common.

The judge receives the generic KG G_D (as a QLever endpoint and as a file), the queries Q, and the output KG G. It is asked to first explore G_D and form its own view of how the data should be modeled, then read Q in full, and then review Q against nine criteria:

C1. Entity and class modeling: the right real-world things become entities, at the right

granularity.

C2. Vocabulary: classes and predicates denote domain concepts rather than source names; each predicate has one precise meaning and a natural direction.

C3. IRI design: entity IRIs are minted from stable identifiers, in a domain namespace, with proper encoding.

C4. Relationships: the relationships of D are links between entities, without G being a replica of the source schema.

C5. Self description: every appropriateentity has an `rdf:type`, and entities with a natural name have an `rdfs:label`.

C6. Schema coverage: the information of every field of D reaches G.

C7. Soundness: Q produces only content that D supports and does not invent new information.

C8. IRI consistency: the same entity gets the same IRI wherever it is minted or referenced.

C9. Literal typing: literals carry datatypes that match their values.

C1 to C4 correspond to the semantic-modeling criteria of Section 1.2, and C6 to C9 to the fidelity criteria, with C8 covering internal consistency. C5 is added because types and labels are what makes a graph navigable for a human or an LLM. Answerability is not judged here, but evaluated with SPARQL queries using the next level of analysis.

The judge assigns each issue a severity. An issue is

- **major** if it loses or corrupts information, makes the graph misleading, or makes an important part of the domain wrong or hard to query;

- **minor** if the information is still present and correct, but less clear, less consistent, or

less convenient than it could have been.

Each severity decision must be justified in one sentence. Each criterion then receives one of five levels:

Level	Anchor
1	No issues.
0.75	Only minor issues.
0.5	One major issue, or minor issues so widespread that they affect most of the graph.
0.25	Several major issues, but the criterion is still partly met.
0	The criterion is essentially not met.

Table 3: Levels of the judge per criterion.

The levels of the nine criteria are summed to a score out of 9, which we report scaled to 10. The judge writes its result as a JSON file, with the issues, their evidence and severity, and a rationale for each level, explaining why it chose this level and not a neighboring one. Problems that fit none of the criteria are reported separately and not scored.

The judge LLM that we use is `DeepSeek-V4.1-Flash`, run as an OpenCode agent, with one judging run per output KG. The prompt is identical for all runs, except for a configuration block that provides the path for the input and output files. The judge is not given the prompt of the generation or review agents, and it does not see any of the intermediate queries or trace information that were generated during modeling.

Qualitative analysis: The third level is a manual analysis of selected runs. As a preliminary step, we use Protégé to visualize and analyze the resulting KGs. For IMDb, Mondial, and MusicBrainz, we then assess usability with analytical questions. For each question, we write one SPARQL query per output KG. We trace every difference between them back to a modeling decision in the output KG, such as an extra hop through an intermediate node, a cast needed because of a datatype, or an IRI pattern that the user must know in order to filter. In this way, the comparison shows not only which graph is simpler and faster to query, but which specific modeling choice causes the difference. For the runtime measurements, each query is run five times on QLever after one warm-up run, with the result cache cleared before every run, and we report the median of QLever's total query time (computation and serialization).

For the five BIRD databases, the analysis focuses on the effect of the competency

questions. For each approach, we compare the run with CQs to the run without CQs on the same database. We identify where the two models differ, for example an entity that is introduced or left out, a field that is modeled differently, or a part of the schema that is no longer covered. We then check whether each difference can be traced back to a specific question. This shows how and where the CQs influenced the modeling, and whether they broadened the model or narrowed it towards what the questions ask.

5.3 Results

5.3.1 Syntactic Validity

All 25 runs that produced queries yield a valid output KG. For the GRASP runs, this holds by construction: every saved query has been executed on the QLever endpoint of G_D during generation, and only queries whose latest test succeeded can be submitted (Section 4.3). All thirteen GRASP query sets were therefore materialized on QLever without changes.

For MusicBrainz, the baseline produced no query set at all. In each of five attempts, the OpenCode session kept exploring the generic KG and then stalled without writing any queries. The same happened once for Mondial, where a later attempt succeeded.

The baseline cannot execute its queries on a SPARQL endpoint, and six of its twelve query sets did not run on QLever as written:

- **Very wide queries.** The baseline tends to write one query per table, with one OPTIONAL per column. The queries for the widest tables, `cards` in `card_games` and `schools` in `california_schools`, contain up to 72 OPTIONALs in a single query. On these, QLever crashed with a stack overflow in its query planner, or did not finish within 600 seconds. GRASP never hit this problem, since a query that crashes the endpoint fails its test and has to be split by the agent.

- **Predicates built at runtime.** The lineup queries of `european_football_2` compute their predicate with `BIND(IRI(CONCAT(...)))` and use the variable in the predicate position, which QLever 0.6.0 rejects with an internal assertion.

- **Unsupported functions.** The IMDb baseline types its years with the cast function `xsd:gYear()`, which the engines we tried do not support: QLever does not support and rejects with an error, Oxigraph also rejects the query, and RDFLib evaluates it to unbound, so the year triples silently disappear. We replaced the cast with the

equivalent `STRDT(STR(?y), xsd:gYear)`.

- **Unencoded IRIs.** The IMDb baseline mints IRIs from the values of `title.akas/types` without encoding them. Ten of these values contain the separator character `0x02` (Section 5.1), which is not allowed in an IRI. We percent-encoded these IRIs in the output.

We materialized the five query sets with the first two problems with Oxigraph instead of QLever, which executes them unchanged, and fixed the IMDb queries as described. None of these interventions changes the modeling of the queries, so the judge evaluates the queries as the baseline wrote them.

Wherever the baseline chose to test its queries in any of the experiments, it did so with rdflib on the sampled N-Triples file, where a query that crashes, times out, or is rejected by QLever can still run, and where an unsupported cast such as `xsd:gYear()` does not fail but silently drops its values. Its self-reported verification therefore did not guarantee that the queries produce the triples the agent claims, and some of the output required interventions before it could be materialized.

5.3.2 LLM-as-a-Judge Scores

Table 4 shows the score of every run on IMDb, Mondial, and MusicBrainz, and Table 5 on the five BIRD databases. Table 6 shows the mean level of each criterion per configuration on the BIRD databases, and the levels of all runs are listed in Table 7.

Dataset	Baseline	GRASP
IMDb	8.6 (2)	**9.2** (0)
Mondial	8.6 (2)	**8.9** (1)
MusicBrainz	–	6.7 (4)

Table 4: Judge score per run on IMDb, Mondial, and MusicBrainz, out of 10, with the number of major issues in parentheses. The best score per dataset is in bold. The baseline produced no output for MusicBrainz.

GRASP versus Baseline (RQ2): Since the baseline produced no output for MusicBrainz, only the GRASP run is scored there, and the comparison covers the remaining seven datasets. Without competency questions, GRASP scores at least as high as the baseline on all seven, and higher on six of them; on superhero, both reach 8.9. The mean over the BIRD databases is 8.5 against 7.6, and the GRASP runs contain far fewer major issues: 3 against 16 on the BIRD databases, and 4 against 20 on all seven datasets.

Dataset	Baseline	Baseline + CQ	GRASP	GRASP + CQ
superhero	8.9 (1)	**9.2** (0)	8.9 (0)	8.9 (0)
card_games	7.2 (4)	7.5 (3)	**7.8** (2)	7.5 (2)
formula_1	7.2 (4)	8.3 (3)	**9.2** (0)	8.9 (1)
european_football_2	7.8 (3)	7.8 (3)	**8.6** (0)	7.8 (2)
california_schools	6.7 (4)	7.5 (3)	8.1 (1)	**8.3** (0)
Mean (BIRD)	7.6	8.1	**8.5**	8.3

Table 5: Judge score per run on the BIRD databases, out of 10, with the number of major issues in parentheses. The best score per dataset is in bold.

Configuration	C1	C2	C3	C4	C5	C6	C7	C8	C9	Score
Baseline	0.65	0.60	0.90	0.55	0.75	0.85	0.80	0.90	0.80	7.6
Baseline + CQ	0.60	0.65	1.00	0.65	0.75	0.95	0.85	0.90	0.90	8.1
GRASP	0.80	0.75	0.90	0.80	0.90	0.85	0.85	0.90	0.90	8.5
GRASP + CQ	0.65	0.65	0.90	0.85	0.85	0.90	0.85	0.95	0.85	8.3

Table 6: Mean level per criterion over the five BIRD databases.

The difference is concentrated in the modeling criteria. Table 6 shows that:

- **Entity and class modeling (C1).** GRASP better identifies the right real-world things to model as entities, with a mean level of 0.80 against 0.65 for the baseline.

- **Vocabulary (C2).** GRASP uses more meaningful predicates and classes, with a mean level of 0.75 against 0.60 for the baseline.

- **Relationships (C4).** The baseline often keeps relationships as literals, while GRASP models them as links between entities. The mean level of C4 is 0.55 for the baseline and 0.80 for GRASP.

- **Self description (C5).** GRASP provides more complete typing and labeling of entities, with a mean level of 0.90 against 0.75 for the baseline.

The scores indicate that IRI design, coverage, and IRI consistency are on par. The judge thus finds that both approaches carry the data over equally completely, but GRASP models it slightly better.

Effect of competency questions (RQ3): From the scores, it appears that competency questions affect the two approaches in opposite directions. For the baseline, adding

CQs never lowers the score: it improves it on four of the five databases, and the mean rises from 7.6 to 8.1. For GRASP, adding CQs lowers the score on three databases, leaves `superhero` unchanged, and improves only `california_schools`. The mean here drops from 8.5 to 8.3. The general trend is that with CQs, the two approaches are close while baseline without CQs is the weakest configuration.

The mean score per criterion over the five BIRD databases as shown in Table 6 shows that for GRASP, adding the CQs lowers the score for entity modeling (C1: 0.80 to 0.65) and vocabulary (C2: 0.75 to 0.65). We interpret this as the agent narrowing its model towards what the questions ask, despite the instruction that the questions are a minimum, not the scope of the model. For the baseline, CQs mainly improve coverage (C6: 0.85 to 0.95) and literal typing (C9: 0.80 to 0.90), but they do not improve its overall modeling of entities.

Schema size and complexity (RQ4): Between IMDb, Mondial, and MusicBrainz, the underlying schema size increases from 14 to 47 to 239 tables. The judge scores IMDb and Mondial similarly, with 8.6 and 8.9, but scores MusicBrainz much lower, with 6.7 with four major issues. Among the other datasets, the score does not simply follow the number of tables: Mondial, with 47 tables and the densest foreign-key structure, scores 8.9, while `card_games`, with six tables, scores lower.

While the number of tables is a factor in the complexity of modeling since the score drops with Musicbrainz, there is no general trend within the scale of under 50 tables that can show that a larger schema is necessarily harder to model correctly. It could potentially be the case that the complexity arises from the domain-specific challenges as well and how far the domain is from the underlying schema i.e. whether the important things of the domain correspond to tables and columns or have to be recognized by the agent.

For instance, in datasets such as `card_games` and `european_football_2`, there is ambiguity in how the data should be interpreted and modeled. Mondial, in contrast, has comparatively more tables, but each of them corresponds closely to a concept of the domain, such as a country, a river, or a membership, so its size adds work but little ambiguity. However, scale does matter as with 239 tables in MusicBrainz, the agent has to cover a large number of tables in one coherent and internally consistent model.

The judge levels of all runs per criterion, and the corresponding scores are listed in Table 7.

Dataset	Run	C1	C2	C3	C4	C5	C6	C7	C8	C9	Score
IMDb	Baseline	0.50	0.50	1.00	1.00	0.75	1.00	1.00	1.00	1.00	8.6
	GRASP	0.75	0.75	1.00	1.00	0.75	1.00	1.00	1.00	1.00	9.2
Mondial	Baseline	0.50	0.75	1.00	1.00	1.00	1.00	1.00	1.00	0.50	8.6
	GRASP	1.00	0.75	0.75	1.00	0.50	1.00	1.00	1.00	1.00	8.9
MusicBrainz	GRASP	0.50	0.50	0.75	0.50	0.75	0.50	0.75	1.00	0.75	6.7
superhero	Baseline	0.50	0.75	1.00	0.75	1.00	1.00	1.00	1.00	1.00	8.9
	Baseline + CQ	0.75	1.00	1.00	0.75	1.00	0.75	1.00	1.00	1.00	9.2
	GRASP	1.00	0.75	1.00	1.00	0.75	0.75	0.75	1.00	1.00	8.9
	GRASP + CQ	0.75	0.75	1.00	1.00	0.75	1.00	0.75	1.00	1.00	8.9
card_games	Baseline	0.75	0.50	1.00	0.50	0.75	0.50	1.00	1.00	0.50	7.2
	Baseline + CQ	0.50	0.50	1.00	0.50	0.75	1.00	1.00	1.00	0.50	7.5
	GRASP	0.50	0.75	1.00	0.50	1.00	0.75	0.75	1.00	0.75	7.8
	GRASP + CQ	0.50	0.75	1.00	0.50	0.75	1.00	0.75	1.00	0.50	7.5
formula_1	Baseline	1.00	0.25	0.75	0.50	0.50	1.00	1.00	1.00	0.50	7.2
	Baseline + CQ	0.75	0.25	1.00	1.00	0.50	1.00	1.00	1.00	1.00	8.3
	GRASP	1.00	0.75	1.00	0.75	1.00	1.00	1.00	1.00	0.75	9.2
	GRASP + CQ	0.75	0.50	1.00	1.00	1.00	1.00	1.00	1.00	0.75	8.9
european_	Baseline	0.50	0.75	1.00	0.50	0.75	1.00	0.50	1.00	1.00	7.8
football_2	Baseline + CQ	0.50	0.75	1.00	0.50	0.75	1.00	0.50	1.00	1.00	7.8
	GRASP	0.75	0.75	1.00	0.75	1.00	0.75	0.75	1.00	1.00	8.6
	GRASP + CQ	0.50	0.50	0.75	0.75	0.75	0.75	1.00	1.00	1.00	7.8
california_	Baseline	0.50	0.75	0.75	0.50	0.75	0.75	0.50	0.50	1.00	6.7
schools	Baseline + CQ	0.50	0.75	1.00	0.50	0.75	1.00	0.75	0.50	1.00	7.5
	GRASP	0.75	0.75	0.50	1.00	0.75	1.00	1.00	0.50	1.00	8.1
	GRASP + CQ	0.75	0.75	0.75	1.00	1.00	0.75	0.75	0.75	1.00	8.3

Table 7: Judge levels of all runs per criterion, and the score out of 10.

Limitations: Each configuration was run once, and each output was judged once by the same LLM. Therefore, it should be noted that the score differences between two individual runs should not be over-interpreted. The judge LLM scores are to be used to understand the overall trend and to identify the most common issues in both the approaches. The scores are not to be interpreted as an absolute measure of quality, but rather as a relative measure of the two approaches and their configurations.

5.3.3 Qualitative Analysis

As per the third level of our evaluation setup, we perform the usability assessment of the output KGs manually. However, since this would require significant manual effort, we perform this analysis only for the three main datasets: IMDb, Mondial, and MusicBrainz. For the five BIRD databases, we instead look at the major issues that the judge reported for each run.

Examples of major issues on the BIRD databases:

- **Structure kept as literals.** In european_football_2, `Match` has eight text columns (`goal`, `shoton`, `shotoff`, `foulcommit`, `card`, `cross`, `corner`, `possession`) whose values are XML documents listing the events of the match. Both baseline runs keep each of them as one string literal per match, so goals, cards, and fouls never become entities and are never linked to their players and teams. GRASP without CQs models them.

- **Predicates with several meanings.** In formula_1, both baseline runs use a single predicate `f1:position` for the finishing position, the qualifying position, the championship position, and the position in a lap.

- **Wrong entities.** In california_schools, both baseline runs type the ten district-level rows of `schools` as `cs:School`, and additionally mint the same districts under a second IRI, so that each of these districts appears as two entities, one of them a school. The judge counts this as a major issue.

- **Invented content.** In european_football_2, both baseline runs write the index of a lineup slot as `ef:shirtNumber`, although the source contains no shirt numbers.

The GRASP runs have fewer major issues, and they are of a different kind. For instance, in california_schools, GRASP without CQs mints addresses and administrators as blank nodes, which are not stable across executions. With CQs, the issues reflect the

narrowing described in the previous section.

Some problems are shared by both approaches and point to the dataset rather than to the approach. In card_games, all four runs model each printing of a card as `mtg:Card` and never introduce the abstract card that the printings share, so the list of sets in which a card was printed is attached to every printing. This is the main reason why card_games has the lowest scores of the BIRD databases for both approaches.

The following paragraphs present the usability assessment of IMDb, Mondial, and MusicBrainz.

IMDb The two IMDb output KGs, baseline and GRASP, model the same key entities: titles, persons, credits, key title types and alternative titles. Both type each title with a class per title type, link credits to their title and person, link episodes to their series, and label every entity. The baseline is scored 8.6 and the GRASP is scored 9.2. Table 8 shows the key modeling differences between the two.

	Baseline	**GRASP**

Rating	separate `Rating` node
	`?t imdb:hasRating ?r .`
	`?r imdb:averageRating ?v`
		on the title
		`?t imdb:averageRating ?v`

Genre	node with a label
	`?t imdb:hasGenre ?g .`
	`?g rdfs:label ?genre`
		literal
		`?t imdb:genre ?genre`

Profession	`Profession` node `?p imdb:hasProfession profession:actor`	literal `?p imdb:profession "actor"`
Credit category	`Role` node, same labels as the `Profession` nodes `?c imdb:role role:actor`	literal `?c imdb:category "actor"`
Namespace	`http://example.org/imdb/`	`http://imdb.org/`

Table 8: Key Modeling differences between the IMDb output KGs that affect querying.

To compare their usability, we used nine analytical questions, wrote corresponding SPARQL queries, and executed them on the output KGs of the evaluation graph G_D^{pop} (Section 5.1). Table 9 shows the number of triple patterns of each query and its median runtime on QLever.

GRASP needs fewer or equally many triple patterns on every question, and its queries are faster or equally fast on every question.

		Triple patterns		Median time (ms)		
#	**Question**	**Baseline**	**GRASP**	**Baseline**	**GRASP**	**Speed-up**
Q1	All genres, with their frequency and average rating	4	2	24	12	2.00
Q2	Persons with the most votes on average (at least 3 movies)	6	5	38	35	1.09
Q3	Persons known for ≥ 3 movies with ≥ 1M votes	5	4	14	12	1.17
Q4	Oldest directors at the time of the movie	5*	5	14	13	1.08
Q5	Series with ≥ 1M votes, by number of episodes	5	4	17	13	1.31
Q6	Oldest actors in movies with ≥ 100K votes	9*	8	131	77	1.70
Q7	Highest-rated movie (≥ 100K votes) per director, top 10	12	10	95	47	2.02
Q8	Youngest directors with a movie over 1M votes	7*	6	22	17	1.29
Q9	Movies whose title contains “star” and “war”	2	2	11	11	1.00
	Total	**55**	**46**	**366**	**237**	**1.54**

Table 9: Analytical queries on the two IMDb output KGs: number of triple patterns and median of five cold-cache runs on QLever. * additionally needs two casts.

Ratings on the title. In D, `title.ratings` is in a one-to-one relationship with `title.basics`, and holds only the average rating and the number of votes. Baseline keeps it as a separate `Rating` node and GRASP attaches both values to the title. So, every question that filters, groups, or ranks by votes or rating, needs one more join in baseline, which applies to seven of the nine questions. In Q7, which looks up the rating twice, once to find the maximum per director and once to find the movie with that maximum, baseline pays this join twice and takes twice as long:

```
# GRASP
?m a imdb:Movie ; imdb:director ?d ; imdb:averageRating ?r ; imdb:numVotes ?v .
# Baseline
?m a tt:movie ; imdb:director ?d ; imdb:hasRating ?rt .
?rt imdb:averageRating ?r ; imdb:numVotes ?v .
```

Years as integers. The baseline types years as `xsd:gYear`, GRASP as `xsd:integer`. The type of the baseline is the more precise one, but SPARQL defines no arithmetic on `xsd:gYear`. In the three questions that compute an age (Q4, Q6, Q8), the natural expression `?y - ?b` silently returns no value on the baseline, without an error, and the query has to cast both years through a string:

```
# GRASP
SELECT ?name (MAX(?y - ?b) AS ?age) WHERE { ... }
# Baseline
SELECT ?name (MAX(xsd:integer(STR(?y)) - xsd:integer(STR(?b))) AS ?age) WHERE { ... }
```

A user who writes the natural query on the baseline receives an empty result, not a wrong one, and would have to know the datatype to find the reason.

Catgeorical values as nodes. The baseline turns genres and credit categories into nodes with their own type and label and GRASP keeps them as literals. Nodes seem to be the more expressive choice, since further information could be attached to a genre, but none of the nine questions benefits from it. Instead, the baseline needs one more hop to reach the name of a genre (Q1), and the user has to know the IRI pattern of the categories (`role:actor`) to filter by them (Q6).

We also note that two node types overlap as well. The baseline builds `Profession` nodes from `primaryProfession` in `name.basics` and `Role` nodes from `category` in `title.principals`, two columns with largely the same values: 11 of the 12 categories, such as `actor` and `director`, also occur as professions. An actor is therefore linked to `profession:actor` directly and to `role:actor` through each credit, two unconnected nodes with the same label. A user who looks up “actor” by its label finds both, and has to know that only the `Role` node says who acted in a given title. The name `Role` is misleading as well, since it suggests the character that is played, which the

baseline stores separately as `imdb:character`. GRASP keeps both values as literals on predicates named after their columns, `imdb:profession` and `imdb:category`, so the two meanings stay apart.

Equal cases. For keyword search over titles (Q9), both graphs keep the primary title as `rdfs:label`, so the queries are identical and equally fast. Neither graph defines a class hierarchy, so "series" has to be expressed as two classes (`tvSeries` and `tvMiniSeries`) in both (Q5), and "actor" as two categories (`actor` and `actress`) in both (Q6).

The absolute differences are small, at most 54 ms on a graph of 2.5 million triples. We only note that the queries on the output KG from GRASP is never slower, and the largest speed-ups occur in the questions with the most additional hops (Q1, Q6, Q7). Together with the judge scores, this shows that the more compact modeling of GRASP is not only judged better, but also makes the graph simpler and faster to query for the set of analytical questions we used.

Mondial The two Mondial output KGs of the baseline and GRASP model the same entities: countries, provinces, cities, organizations, and geographical features, with reified nodes for borders, memberships, and population observations. Both type and label their main entities. The judge scores them 8.6 and 8.9 respectively. The key differences are listed in Table 10.

	Baseline	**GRASP**

Economy, population, politics	separate node per table
	`?e mond:country ?c ;`
	`mond:gdpMillionUSD ?g`
		on the country
		`?c ex:gdpMillionUSD ?g`

Continent, organization, language	reified node only
	`?m mond:country ?c ;`
	`mond:organization ?o`
		reified node and shortcut
		`?c ex:memberOf ?o`

Borders	`Border` node and direct edge
	`?c mond:borders ?n`
		`Border` node only
		`?c ex:hasBorder ?b .`
		`?b ex:country1 ?n`

Location of features	one predicate per role
	`mond:sourceInProvince`
	`mond:estuaryInProvince`
	`mond:flowsThroughProvince`
		one predicate for all roles
		`ex:province`
		`ex:country` (direct)

Religion	name as a literal on the share `?a mond:religionName ?n`	shared `Religion` node `?a ex:religion ?r`
Years	`xsd:gYear`	`xsd:integer`

Table 10: Modeling differences between the Mondial output KGs.

Since Mondial is small, we executed both query sets on the generic KG of the complete data (329,932 triples). We used eleven analytical questions, wrote corresponding SPARQL query per graph and executed them on both graphs. Table 11 shows the number of triple patterns and the median runtime on QLever for each query.

		Triple patterns		Median time (ms)		
#	**Question**	**Baseline**	**GRASP**	**Baseline**	**GRASP**	**Speed-up**
Q1	Countries with the highest GDP per capita	4	3	7	8	0.88
Q2	Organizations with members on all six continents	5	3	10	11	0.91
Q3	Organizations whose members speak the most languages	5	3	16	13	1.23
Q4	Rivers through the most countries	4	3	7	6	1.17
Q5	Countries with the most mountains over 4,000 m	5	4	12	11	1.09
Q6	Landlocked countries	4	4	4	6	0.67
Q7	Largest average population increase per year	12*	12	21	28	0.75
Q8	Countries independent since 1990, by GDP, with unemployment	6	4	14	11	1.27
Q9	Total land-border length per country	3	3	10	7	1.43
Q10	Countries with the most neighbors	2	3	3	16	0.19
Q11	All countries reachable over land from France	2	2	3	13	0.23
	Total	**52**	**44**	**107**	**130**	**0.82**

Table 11: Analytical queries on the two Mondial output KGs. Speed-up is the time of the baseline divided by the time of GRASP; values below 1 mean that the baseline is faster. * additionally needs two casts.

GRASP needs fewer triple patterns on six questions and the baseline on one. On runtime, the baseline is faster on six of the eleven questions. Most of these differences are 1–3 ms. The only large differences are Q10 and Q11, where the baseline is four to five times faster. In contrast to IMDb, neither graph is better on all questions; each is better where it has materialized a shortcut for that particular question.

Country statistics as properties of the country. In D, `Economy`, `Population`, and `Politics` are in a one-to-one relationship with `Country`. The baseline keeps each as a separate node that points to the country through `mond:country`, GRASP attaches

their values to the country. GRASP also adds direct shortcuts from the country to its continents, organizations, and languages, next to the reified membership nodes. Every question that starts from a country therefore needs one or two patterns less in GRASP (Q1–Q5, Q8). For instance, Q8 needs the independence date from `Politics` and the GDP and unemployment rate from `Economy`. GRASP reads all three from the country, while the baseline needs one node per table:

```
# GRASP
?c ex:independenceDate ?d ; ex:gdpMillionUSD ?gdp ; ex:unemploymentPercent ?u .
# Baseline
?pi mond:independence ?d ; mond:country ?c .
?e mond:gdpMillionUSD ?gdp ; mond:unemploymentPercent ?u ; mond:country ?c .
```

On QLever, the additional joins cost little: GRASP is at most 1.3 times faster on these questions, and on Q1 and Q2, the baseline is even slightly faster.

Borders between countries. The baseline materializes the symmetric relation `mond:borders` between two countries, in addition to the `Border` node that holds the border length. GRASP keeps only the `Border` node, which names the two countries as `ex:country1` and `ex:country2`. To reach a neighbor in GRASP, a query has to go through the node, try both sides, and remove the start country. This is the largest difference in the set: the baseline is 5.3 times faster on Q10, and 4.3 times faster on Q11, where the path is repeated in a transitive closure:

```
# Baseline
<.../country/F> mond:borders+ ?x .
# GRASP
<.../resource/country/F> (ex:hasBorder/(ex:country1|ex:country2))+ ?x .
```

Where the question is about the border itself, as in Q9 (border length per country), the effect is reversed: GRASP reaches the borders of a country directly through `ex:hasBorder` and is 1.4 times faster.

Merged feature locations. In D, a river is linked to provinces through three tables: `geo_Source`, `geo_Estuary`, and `geo_River`. The baseline keeps the three roles as separate predicates. GRASP reads all `geo_*` tables in one query through a variable predicate, so all three become the same `ex:province` edge, and GRASP adds the country of each province directly to the feature. This gives GRASP a one-hop path from a feature to its countries (Q4, Q5), but it removes the distinction between source and mouth.

Years and observation classes. As on IMDb, the baseline types years as `xsd:gYear`, so the yearly increase in Q7 needs `xsd:integer(STR(?y))` on both years. GRASP

uses `xsd:integer` and needs no cast. The baseline is still faster on Q7 (21 ms against 28 ms), because it has a separate class for the population observations of countries, while GRASP uses one class for countries, provinces, and cities and has to select the country observations through their subject.

Overall, the GRASP graph is simpler to query whenever a question starts from a country, since the statistics are properties of the country and memberships have direct shortcuts. The baseline graph, on the other hand,is faster for questions about the network of countries. It should also be noted that the merged feature locations indicate that a more compact model may also lose a distinction that a user might need. Considering the close judge scores as well, this shows that on Mondial, both approaches produce graphs of comparable usability, with different trade-offs.

MusicBrainz For MusicBrainz, the baseline produced no output, so we compare the GRASP output KG with the generic KG G_D itself. GRASP models artists, releases, recordings, works, labels, areas, and their relationships, with artist credits as separate nodes. The judge scores it 6.7, with four major issues: editor collections are not modeled, the IPI and ISNI identifiers of labels never reach the graph, and the direct relationship predicates are named after the name of the link type, which is reused for different meanings and is emitted in the wrong direction for many link types. The key differences to G_D are listed in Table 12.

	G_D **(generic KG)**	**GRASP**

Relationship	row of `l_artist_artist`
	`?l r:link ?k .`
	`?k r:link_type ?lt .`
	`?lt lt:name "member of band"`
		direct edge and reified node
		`?m rel:member_of_band ?b`

Attributes	one predicate per table `artist:name`, `ref:artist/area`	shared predicates `rdfs:label`, `mb:area`
Dates	year, month, and day as integers	one literal, typed by precision `xsd:gYear`, `xsd:gYearMonth`, or `xsd:date`
ISO code	separate `iso_3166_1` row	on the area `mb:iso3166_1`
Release country	through `country_area` `release_country` → `country_area` → `area`	on the release event `?event mb:country ?area`

Table 12: Modeling differences between the MusicBrainz generic KG and output KG that affect querying.

As for IMDb, the sample is not suited for analytical questions: the artist table is sampled,

so counts of artists per area or of members per band are incomplete. We therefore built a connected test graph from the complete dump with the same converter. It contains all rows of the 14 tables that the questions use: areas and their hierarchy, artists and their relationships, releases, release events, media, and the link rows and link types that the relationships use. It has 418,384 rows and 5,643,288 triples in the layout of G_D. Executing the queries of GRASP on it with QLever gives 2,665,382 triples. We wrote eleven analytical questions, each as one SPARQL query per graph. All 11 queries on G_D and 10 of the 11 on GRASP return the same answer as the gold SQL query; Q8 on GRASP misses 19 of 28,018 rows (see below). Table 13 shows the number of triple patterns and the median runtime on QLever, measured as for IMDb.

		Triple patterns		**Median time (ms)**		
#	**Question**	G_D	**GRASP**	G_D	**GRASP**	**Speed-up**
Q1	Bands with the most members	6	2	41	19	2.16
Q2	Youngest band members at the time they joined	9	8*	52	36	1.44
Q3	Countries (ISO code) with the most artists	3	3	12	23	0.52
Q4	Subdivisions that directly contain the most cities	10	6	46	15	3.07
Q5	Subdivisions whose counties contain ≥ 3 cities	18	9	359	29	12.38
Q6	Countries with the most areas inside them, at any depth	4	4	13	12	1.08
Q7	Countries with the most artists, including sub-areas	5	6	29	523	0.06
Q8	Persons with the longest lifespan	5	5*	24	38†	0.63
Q9	Groups founded per decade	3	3*	13	18	0.72
Q10	Countries with the most release events	3	2	9	7	1.29
Q11	Releases with the most tracks	3	3	16	22	0.73
	Total	**69**	**51**	**614**	**742**	**0.83**

Table 13: Analytical queries on the MusicBrainz generic KG and output KG, executed on the connected test graph: number of triple patterns and median of five cold-cache runs on QLever. Speed-up is the time of G_D divided by the time of GRASP; values below 1 mean that G_D is faster. * additionally needs a cast of each date. † incomplete answer.

GRASP needs fewer triple patterns on five questions and more on one, 51 against 69 in total. It is faster on six questions, but slower in total, 742 ms against 614 ms, because of Q7 alone; without Q7, GRASP takes 219 ms against 585 ms. The differences trace back to four modeling decisions.

Direct relationship edges. In G_D, a relationship is a row of an `l_*` table, and its type is reached through the `link` row and its `link_type`. GRASP keeps this row as a reified node, but additionally emits a direct edge between the two entities, named after the link type. A question about a typed relationship therefore becomes a single pattern:

```
# GRASP
?m rel:member_of_band ?b .
# G_D
?lt lt:name "member of band" . ?k r_link:link_type ?lt .
?l r_lar:link ?k ; r_lar:entity0 ?m ; r_lar:entity1 ?b .
```

This saves the most where a question follows several relationships. Q5 goes down the area hierarchy twice, and GRASP needs half the patterns and is 12 times faster. On Q1 and Q4, GRASP is two to three times faster. Q2 also needs the date on which the member joined, which is a property of the relationship, so GRASP has to use the reified node and saves only one pattern.

Direction of the relationship edges. The direct edge always goes from the first entity of a relationship row to the second, and is named after the `name` of the link type, not after its directional `link_phrase`. For areas, the first entity is the parent, so GRASP states that `"United States" rel:part_of "Virginia"`. The queries for the area questions (Q4–Q7) therefore have to be written backwards, against the meaning of the predicate:

```
# GRASP: the subdivision ?s "is part of" the city ?c
?s rel:part_of ?c ; mb:areaType ?st . ?st rdfs:label "Subdivision" .
```

The natural query, `?city rel:part_of ?state`, finds one pair of a city and a subdivision instead of 805, without an error. This is the major issue that the judge reports under C4. The answers in Table 13 are correct only because we inverted the direction.

Shared predicates. GRASP uses one predicate for the same attribute of different entity types, such as `mb:area` for artists, labels, and places, and `rdfs:label` for all entities. A query on artists then needs an additional type pattern, and the engine has to scan more triples. This makes the lookups Q3, Q9, and Q11 1.4 to 1.9 times slower than on the table-specific predicates of G_D. In Q7, which combines the area of an artist with the transitive area hierarchy, GRASP is 18 times slower. Most of this is caused by the plan that QLever chooses for a transitive path that is joined from both sides: when the closure is computed in a subquery first, the same question takes 30 ms on G_D and 37 ms on GRASP.

Dates as one literal. G_D keeps the year, month, and day of a date as three integer columns. GRASP combines them into one literal, whose type depends on the precision:

`xsd:gYear`, `xsd:gYearMonth`, or `xsd:date`. Every question that computes with years must first cut the year out of the string (Q2, Q8, Q9):

```
# G_D
SELECT ?name ((?died - ?born) AS ?years) WHERE { ... }
# GRASP
SELECT ?name ((xsd:integer(SUBSTR(STR(?died), 1, 4))
              - xsd:integer(SUBSTR(STR(?born), 1, 4))) AS ?years) WHERE { ... }
```

GRASP also chooses the type by the length of the combined string (4, 7, or 10 characters). A year with fewer than four digits gives a string of a different length, and the whole date is dropped. In the test graph, this affects 26 artists, and 19 rows of the answer to Q8 are missing.

Equal cases. For the release events per country (Q10), GRASP links the event directly to the area and skips the `country_area` table of G_D, which saves one pattern. For the areas inside each country (Q6), both graphs need one transitive path and are equally fast.

Overall, GRASP is simpler and faster to query than G_D for the part of MusicBrainz that G_D makes hardest: typed relationships, which need two additional hops per relationship in G_D. It is not uniformly better, however. Its shared predicates make lookups slower, its date encoding forces casts and loses dates, and its reversed relationship edges return wrong answers to the natural query without an error. This is consistent with the low judge score: its modeling makes the central relationships easy to query, but its errors are of a kind that a user cannot detect from the results.

6 Conclusion

In this thesis, we investigated to what extent an LLM agent can automate the semantic modeling of structured data by generating SPARQL CONSTRUCT queries. Our approach converts a dataset into a generic KG G_D and lets a GRASP generation agent remodel it, testing every query on G_D, with an independent review agent checking the result. We compared it with the general-purpose coding agent OpenCode on eight datasets with 3 to 239 tables.

RQ1: For schemas of up to 47 tables, the agent produces good semantic models. All thirteen GRASP query sets executed on QLever without changes, and the judge scores of all GRASP runs on these datasets range from 7.5 to 9.2 out of 10, with at most two major issues per run.

RQ2: Without competency questions, our approach scores at least as high as the baseline on all seven datasets where both produced a result, and higher on six. On the BIRD databases, the mean is 8.5 against 7.6, with 3 major issues against 16, and the gains lie in relationships, entity modeling, vocabulary, and self description. Six of the twelve baseline query sets did not run on QLever, although the baseline reported them as tested. On IMDb, the GRASP graph is also simpler and faster to query on every analytical question. On Mondial, the two graphs trade shortcuts, and neither is better on all questions.

RQ3: Competency questions raise the baseline mean from 7.6 to 8.1, mainly through better coverage and typing, but lower the GRASP mean from 8.5 to 8.3, because the agent narrows its model towards the questions. They help an agent that otherwise misses data, and can harm one that already explores it thoroughly.

RQ4: The score does not follow the number of tables: Mondial (47 tables) scores 8.9, card_games (6 tables) lowest among the BIRD databases. Datasets are hard when their domain is far from their schema. Only MusicBrainz, with 239 tables, drops to 6.7. The characteristic failure modes are:

- relationship predicates that are reused for different meanings or point in the wrong direction;

- parts of a large schema that are never read;

- datatypes that force casts or silently drop values, such as `xsd:gYear` years and dates built as strings;

- predicates shared across entity types, and merged roles that remove distinctions a user needs.

These errors produce well-formed triples and become visible only when the output is queried.

Limitations: Each configuration was run and judged once, and the judge is the same LLM as the generators, DeepSeek V4.1 Flash, which is the only model we used. The agents only see samples of the data. The analytical questions were written by us, and the held-out competency questions have not yet been used to measure answerability.

Future work:

- **Review on the output.** Letting the reviewer query the materialized output, for example with the competency questions, would expose wrong directions and dropped values that the queries do not show.

- **Questions as tests.** Giving competency questions only to the reviewer could keep their benefit without narrowing the model.

- **Large schemas.** Modeling large schemas in parts, with a consistency check across parts.

- **Broader evaluation.** Repeated runs, further LLMs, and answerability on the held-out questions.

Overall, an LLM agent that explores and tests against the data produces good semantic models for schemas of up to a few dozen tables, and better ones than a general-purpose agent. Since its output is a set of CONSTRUCT queries, a human expert can find and correct the errors that remain.

List of Figures

1 Three tables of the IMDb dataset with selected columns. 2
2 The generic KG of the IMDb tables (top) and a semantic KG of the same data (bottom). 3
3 The query “Who directed *Heat*?” over the generic KG (bottom left) and over the semantic KG (bottom right). 3
4 A CONSTRUCT query over a sample RDF KG 13
5 GRASP sample shape format for one class 17
6 GRASP sample structural notes 17
7 Overview of generic-to-semantic KG pipeline 18
8 Generation and review of CONSTRUCT queries. 23

List of Tables

1 Rules of the direct mapping 20
2 Overview of the datasets 31
3 Levels of the judge per criterion. 39
4 Judge scores on IMDb, Mondial, and MusicBrainz 41
5 Judge scores on the BIRD databases 42
6 Mean judge level per criterion 42
7 Judge levels of all runs 44
8 Key Modeling differences between the IMDb output KGs that affect querying. 46
9 Analytical queries on the IMDb output KGs 47
10 Modeling differences between the Mondial output KGs. 49
11 Analytical queries on the Mondial output KGs 50
12 Modeling differences between the MusicBrainz generic KG and output KG that affect querying. 52
13 Analytical queries on the MusicBrainz KGs 53

Bibliography

[1] Anomaly Innovations and OpenCode contributors. OpenCode: The open source AI coding agent. https://opencode.ai, source code at https://github.com/anomalyco/opencode, 2025.

[2] Marcelo Arenas, Alexandre Bertails, Eric Prud'hommeaux, and Juan Sequeda. A direct mapping of relational data to RDF. W3c recommendation, World Wide Web Consortium (W3C), September 2012. URL https://www.w3.org/TR/2012/REC-rdb-direct-mapping-20120927/. Latest version: https://www.w3.org/TR/rdb-direct-mapping/.

[3] Julián Arenas-Guerrero, David Chaves-Fraga, Jhon Toledo, María S. Pérez, and Oscar Corcho. Morph-KGC: Scalable knowledge graph materialization with mapping partitions. *Semantic Web*, 15(1):1–20, 2024.

[4] Luigi Asprino, Enrico Daga, Aldo Gangemi, and Paul Mulholland. Knowledge graph construction with a façade: A unified method to access heterogeneous data sources on the web. *ACM Transactions on Internet Technology*, 23(1), 2023.

[5] Hannah Bast and Björn Buchhold. QLever: A query engine for efficient SPARQL+Text search. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*, pages 647–656, 2017.

[6] Haonan Bian. Llm-empowered knowledge graph construction: A survey. *arXiv preprint arXiv:2510.20345*, 2025.

[7] Carla Castedo, Enrique Iglesias, Manuel Lama, Alberto Bugarin-Diz, Maria-Esther Vidal, and David Chaves-Fraga. Blinkg: A benchmark for llm-integrated knowledge graph generation. *arXiv preprint arXiv:2605.19518*, 2026.

[8] Richard Cyganiak. Tarql: SPARQL for tables. https://github.com/tarql/tarql. Accessed: 2026-10-09.

[9] Souripriya Das, Seema Sundara, and Richard Cyganiak. R2RML: RDB to RDF mapping language. W3C Recommendation, 2012. URL https://www.w3.org/TR/r2rml/.

[10] Anastasia Dimou, Miel Vander Sande, Pieter Colpaert, Ruben Verborgh, Erik Mannens, and Rik Van de Walle. RML: A generic language for integrated RDF mappings of heterogeneous data sources. In *Proceedings of the Workshop on Linked Data on the Web (LDOW)*, 2014.

[11] Mariano Fernández-López, Asunción Gómez-Pérez, and Natalia Juristo. METHONTOLOGY: From ontological art towards ontological engineering. In *Proceedings of the AAAI-97 Spring Symposium on Ontological Engineering*, number SS-97-06 in AAAI Technical Report, pages 33–40, Stanford, CA, USA, 1997. AAAI Press.

[12] Johannes Frey, Lars-Peter Meyer, Natanael Arndt, Felix Brei, and Kirill Bulert. Benchmarking the abilities of large language models for rdf knowledge graph creation and comprehension: how well do llms speak turtle? *arXiv preprint arXiv:2309.17122*, 2023.

[13] Michael Grüninger and Mark S. Fox. Methodology for the design and evaluation of ontologies. In *Proceedings of the IJCAI-95 Workshop on Basic Ontological Issues in Knowledge Sharing*, 1995.

[14] Ali Hariri, Stéphane Jean, and Mickaël Baron. Towards automating rdf extraction for archaeological knowledge graphs with llms. In *International Conference on Database and Expert Systems Applications*, pages 83–97. Springer, 2025.

[15] Steve Harris and Andy Seaborne. SPARQL 1.1 query language. W3C Recommendation, March 2013. URL `https://www.w3.org/TR/sparql11-query/`.

[16] Marvin Hofer, Johannes Frey, and Erhard Rahm. Towards self-configuring knowledge graph construction pipelines using llms-a case study with rml. In *Fifth International Workshop on Knowledge Graph Construction@ ESWC2024*, 2024.

[17] IMDb.com, Inc. IMDb non-commercial datasets. `https://data.imdb.com/non-commercial-datasets/`, 2026. Files downloaded from `https://datasets.imdbws.com/` on <YYYY-MM-DD>. Personal and non-commercial use only. Accessed: <YYYY-MM-DD>.

[18] Vamsi Krishna Kommineni, Birgitta König-Ries, and Sheeba Samuel. From human experts to machines: An LLM supported approach to ontology and knowledge graph construction. *arXiv preprint arXiv:2403.08345*, 2024.

[19] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2305.03111.

[20] Anna Sofia Lippolis, Miguel Ceriani, Sara Zuppiroli, and Andrea Giovanni Nuzzolese. Ontogenia: Ontology generation with metacognitive prompting in large language models. In *The Semantic Web: ESWC 2024 Satellite Events*, 2024.

[21] Anna Sofia Lippolis, Mohammad Javad Saeedizade, Robin Keskisärkkä, Sara Zuppiroli, Miguel Ceriani, Aldo Gangemi, Eva Blomqvist, and Andrea Giovanni Nuzzolese. Ontology generation using large language models, March 2025. URL https://arxiv.org/abs/2503.05388.

[22] Ioana Ramona Martin, Tudor Cioara, Ionut Anghel, and Gabriel Arcas. Performance evaluation of large language models for automated knowledge graph generation. *Machine Learning with Applications*, page 100923, 2026.

[23] Wolfgang May. Information extraction and integration with FLORID: The MONDIAL case study. Technical Report 131, Universität Freiburg, Institut für Informatik, 1999. Available from http://dbis.informatik.uni-goettingen.de/Mondial.

[24] MetaBrainz Foundation. MusicBrainz database, sample data dump 20261001-000001. https://data.metabrainz.org/pub/musicbrainz/data/sample/20261001-000001/, 2026. TSV export (mbdump, mbdump-derived). Core data CC0 1.0; derived data CC BY-NC-SA 3.0. Accessed: <YYYY-MM-DD>.

[25] Lars-Peter Meyer, Johannes Frey, Desiree Heim, Felix Brei, Claus Stadler, Kurt Junghanns, and Michael Martin. Llm-kg-bench 3.0: A compass for semantic technology capabilities in the ocean of llms. In *European Semantic Web Conference*, pages 280–296. Springer, 2025.

[26] Maria Angela Pellegrino and Gabriele Tuozzo. From quality reports to knowledge graphs: a case study on csv-to-kg transformation. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 1626–1632, 2025.

[27] María Poveda-Villalón, Alba Fernández-Izquierdo, Mariano Fernández-López, and Raúl García-Castro. LOT: An industrial oriented ontology engineering framework. *Engineering Applications of Artificial Intelligence*, 111:104755, 2022.

[28] Alex Randles and Declan O'Sullivan. R2 [rml]-chatgpt framework. In *Fifth International Workshop on Knowledge Graph Construction@ ESWC2024*, 2024.

[29] Mohammad Javad Saeedizade and Eva Blomqvist. Navigating ontology development with large language models. In Albert Meroño Peñuela, Anastasia Dimou, Raphaël Troncy, Olaf Hartig, Maribel Acosta, Mehwish Alam, Heiko Paulheim, and Pasquale Lisena, editors, *The Semantic Web: 21st International Conference, ESWC 2024, Proceedings, Part I*, volume 14664 of *Lecture Notes in Computer Science*, pages 143–161, Cham, 2024. Springer. doi: $10.1007/978\text{-}3\text{-}031\text{-}60626\text{-}7_8$.

[30] Wilma Johanna Schmidt, Irlán Grangel-González, Adrian Paschke, and Evgeny Kharlamov. LLM-supported manufacturing mapping generation. *Transactions on Graph Data and Knowledge*, 3(3):5:1–5:22, 2025. doi: $10.4230/\mathrm{TGDK}.3.3.5$.

[31] Milena Trajanoska, Riste Stojanov, and Dimitar Trajanov. A multi-agent system for semantic mapping of relational data to knowledge graphs. *arXiv preprint arXiv:2511.06455*, 2025.

[32] Gabriele Tuozzo et al. Moving from tabular knowledge graph quality assessment to rdf triples leveraging chatgpt. In *TEXT2KG/DQMLKG@ ESWC*, page 9, 2024.

[33] Mikel Val-Calvo, Mikel Egaña Aranguren, Juan Mulero-Hernández, Ginés Almagro-Hernández, Prashant Deshmukh, José Antonio Bernabé-Díaz, Paola Espinoza-Arias, José Luis Sánchez-Fernández, Juergen Mueller, and Jesualdo Tomás Fernández-Breis. OntoGenix: Leveraging large language models for enhanced ontology engineering from datasets. *Information Processing & Management*, 62 (3), 2025.

[34] Jeffrey S. Vitter. Random sampling with a reservoir. *ACM Transactions on Mathematical Software*, 11(1):37–57, 1985.

[35] Sebastian Walter and Hannah Bast. GRASP: Generic reasoning and SPARQL generation across knowledge graphs. In *The Semantic Web – ISWC 2025: 24th International Semantic Web Conference, Proceedings, Part I*, volume 16140 of

Lecture Notes in Computer Science, pages 271–289, Cham, 2025. Springer.

[36] Guohui Xiao, Lin Ren, Guilin Qi, Haohan Xue, Marco Di Panfilo, and Davide Lanti. LLM4VKG: Leveraging large language models for virtual knowledge graph construction. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, pages 4715–4723, 2025. doi: $10.24963/\mathrm{ijcai}.2025/525$.

[37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Appendix

This appendix lists the prompts of the two agents of our approach (Section 4.3) and of the LLM judge (Section 5.2). Text in braces, such as `{KG_NAME}`, is filled in per run; everything else is reproduced verbatim.

Generation agent: The system prompt is shown with competency questions. Without them, step 4, the two paragraphs after the steps, and the list of competency questions are omitted, and the submission step becomes step 4.

```
You are a semantic RDF generation assistant. Your task is to analyse the "{KG_NAME}"
knowledge graph and produce one or more SPARQL CONSTRUCT queries that model it as a
well-formed, semantically correct knowledge graph. The knowledge graph may already
model some of its data well - where it does, preserve or refine that modelling;
where it doesn't, remodel it properly. The goal is the best possible semantic model
of the input data.

You should follow a step-by-step approach:
1. Explore the knowledge graph - use the available notes (and shape profile if
available) as a starting point, and run targeted queries where needed to fill gaps.
Get a clear picture of what classes and entities/predicates exist, what the values
look like, and what the data is about, and where the existing modelling is already
sound versus where it needs improvement.
2. Design the mapping:
- Decide what entities to model, what type to assign each, and what fields represent
inter-entity relationships.
- Decide how to map them: mint IRIs from a stable identifier of the entity itself,
in a namespace that reflects the domain - replacing the source's if it does not.
- Identify what null looks like in the data and how to handle it in the mapping.
- Identify whether there are unit informations like currency, scale or language
metadata etc. associated with the data and how to handle them in the mapping.-
Decide whether one CONSTRUCT query is enough to cover the data completely, or
whether the relationships in the knowledge graph require multiple queries. There is
no fixed number - let what you find in the knowledge graph decide, but together the
queries must cover all the data without loss.
3. Write each CONSTRUCT query and test it independently using propose_construct
under a short, descriptive name. Every test saves the query text under that name;
testing the same name again replaces the saved text. Check that it runs and produces
non-empty, well-formed output. Do not add a LIMIT yourself - the test applies its
own, and the saved query must cover all the data.
4. Before submitting, check the queries you wrote against each competency question
by sketching the SPARQL query that would answer it against the graph produced by
your queries. Use it as a usability test to see if the model is semantically
well-formed and easy to query. If the question cannot be answered against the model
at all, the design is missing something that has to be fixed. Do not narrow or make
the model redundant to make the competency questions easier to answer.
5. Once every query you planned is ready, call submit_construct with the names of
all queries that make up the set. The saved text of each named query is what gets
submitted.

Treat each competency question listed below as a query someone may later write
against your output knowledge graph, and use it as a semantic usability test of the
model that you propose: if the query answering it is overly complex - that verbosity
of the SPARQL query may point to a gap in the semantic modelling itself that can be
```

```
improved unless the underlying data requires such modeling.

The list of competency questions is not exhaustive - it states the minimum the
output knowledge graph must be able to answer, not the full scope of it. Modelling
all of the source data completely still applies.

## Competency questions

1. {COMPETENCY_QUESTION_1}
2. {COMPETENCY_QUESTION_2}
3. ...
```

GRASP then appends its standard sections on the index types, the available knowledge graphs, and their notes and prefixes, followed by the rules. Rules 1–5 are the general rules of GRASP, rules 6–11 are specific to this task.

```
## Rules to follow

1. Explain your thought process before each step and function call.
2. Do not ask the user for clarification, neither on the initial input nor on
follow-up inputs or feedback. When the task input is incomplete or ambiguous,
proceed based on reasonable assumptions.
3. Use IRIs returned in function call results as is in subsequent function calls.
Shortening them to their prefixed form, and escaping or encoding special characters
might lead to errors and unexpected or empty results.
4. Do not use "SERVICE wikibase:label { bd:serviceParam wikibase:language ..." in
SPARQL queries. It is not SPARQL standard and unsupported by most SPARQL endpoints.
Use rdfs:label or similar properties to get labels instead.
5. If example or shape indices are available, using them early on to quickly find
relevant information to solve the task is recommended.
6. Always add rdfs:label to every entity in the CONSTRUCT but never make the label a
precondition for the entity's other triples - a missing label shouldn't drop the
entity from the query.
7. Where a property may be missing for a particular entity, generate it
independently (e.g. with OPTIONAL) so a missing value doesn't suppress triples for
the other properties of the same entity - but only where the data is actually
missing, not as a blanket pattern for every property.
8. Never emit a triple with a null marker itself as the value.
9. If the same entity type is referenced by more than one query, mint its IRIs using
the exact same pattern in every query that touches it, and reuse the same predicate
names and namespace for the same concept across queries, so the queries combine into
one coherent, non-conflicting graph.
10. A field name may carry information the value itself doesn't - a unit, a
currency, a scale. Keep that information in the model rather than losing it with the
name.
11. When resubmitting after reviewer feedback, change a query by testing the new
version with propose_construct under the same name, then call submit_construct with
the names of all queries in the set. The names you send are the whole set: a name
you leave out is dropped, and its reason goes in the `removed` argument. Use
list_constructs and get_construct to check the saved names and texts instead of
relying on earlier messages.
```

The first user message is:

```
Analyse the "{KG_NAME}" knowledge graph and generate one or more SPARQL CONSTRUCT
queries that model it as a well-formed, semantically correct knowledge graph.
```

Review agent: The system prompt is assembled in the same way. The two competency-question items and the list of competency questions are only included when competency questions are given, and the item `CROSS-QUERY CONSISTENCY` only when more than one query is submitted.

```
You are an independent validator reviewing a set of one or more proposed SPARQL
CONSTRUCT queries. Your job is to verify whether this set correctly transforms the
source knowledge graph into a well-formed semantically correct RDF knowledge graph.
Once you have finished your investigation, call give_feedback with your findings.
The goal is the best possible semantic model of the input data.

Validate the following aspects of the proposed CONSTRUCT queries:
- SOURCE COVERAGE: Verify that all data from the "{KG_NAME}" knowledge graph is
accounted for in the CONSTRUCT queries. Confirm that nothing significant is missing.
- VALUE TYPES AND RELATIONSHIPS: For each mapped predicate, check that the output
value type is appropriate - literals where a literal is expected, IRIs where a
relationship to another entity is intended.
- RELATIONSHIP MODELING: Query the "{KG_NAME}" knowledge graph to identify what
relationships exist between entities in the data. Verify that the CONSTRUCT output
models these relationships appropriately. Any field that semantically references
another entity - whether that entity is minted within this same query or by a
different query in the set - must be resolved as an IRI link to that entity, not
left as a raw literal value.
- CONSISTENCY: Check that the same real-world entity is minted using the same IRI
pattern everywhere it's referenced, whether within a single query or across the
query set.
- NAMESPACE CONSISTENCY: Verify that all custom predicates and entity IRIs use the
same namespace consistently throughout the queries and that this namespace reflects
the intended domain and semantics. Judge this from the expanded IRI, not the prefix
alias.
- COMPETENCY QUESTION COVERAGE: For each competency question listed, write the
SPARQL query that would answer it against the knowledge graph the proposed CONSTRUCT
queries would produce. You cannot execute it, since that graph does not exist yet,
but writing it out is a usability test of the model: if the query answering it is
overly complex and requires additional work to produce the correct result, that
points to a gap in the semantic modelling itself and should be flagged. Flag any
question that could not be answered at all, and say which part of the model is the
reason.
- COMPETENCY QUESTION SCOPE: Competency questions are a minimum, not the full scope.
Confirm the model has not been narrowed or made redundant to accommodate the
questions while sacrificing SOURCE COVERAGE.
- CROSS-QUERY CONSISTENCY: When multiple queries are submitted together, verify that
they form a coherent graph. Check that the same real-world entity is named and
modelled consistently across queries, the same predicate names and namespace are
used for the same relationship across queries, and that any entity type referenced
by more than one query is minted using the same IRI pattern so the same real-world
entity always resolves to the same IRI.

## Competency questions
```

```
1. {COMPETENCY_QUESTION_1}
2. {COMPETENCY_QUESTION_2}
3. ...
```

It is followed by the standard sections of GRASP and the rules:

```
## Rules to follow

1. Explain your thought process before each step and function call.
2. Do not ask the user for clarification, neither on the initial input nor on
follow-up inputs or feedback. When the task input is incomplete or ambiguous,
proceed based on reasonable assumptions.
3. Use IRIs returned in function call results as is in subsequent function calls.
Shortening them to their prefixed form, and escaping or encoding special characters
might lead to errors and unexpected or empty results.
4. Do not use "SERVICE wikibase:label { bd:serviceParam wikibase:language ..." in
SPARQL queries. It is not SPARQL standard and unsupported by most SPARQL endpoints.
Use rdfs:label or similar properties to get labels instead.
5. If example or shape indices are available, using them early on to quickly find
relevant information to solve the task is recommended.
6. Do not trust the query as written - independently verify each aspect by querying
the source knowledge graph directly.
7. Only report issues you have actually verified through knowledge graph
exploration. Only call give_feedback once you have finished your full investigation.
```

The user message contains the submitted queries and the computed source coverage. If the source predicates cannot be listed, the coverage section only states that the coverage is unavailable and asks the reviewer to verify it by querying the knowledge graph.

````
Review the following {N} SPARQL CONSTRUCT queries targeting the "{KG_NAME}"
knowledge graph. Together they are intended to produce one coherent semantic graph:

### {QUERY_NAME_1}
```sparql
{QUERY_1}
```


...

## Computed source coverage

{COVERED} of {TOTAL} source predicates ({PERCENT}%) are referenced by these queries.

A predicate counts as referenced if its IRI appears anywhere in a query, so this is
an upper bound: a predicate can be referenced and still have its values dropped.

The following {N_UNCOVERED} source predicate(s) are referenced by NO query, ordered
by triple count:
````

```
- <{PREDICATE_IRI}> ({COUNT} triples)
- ...

Each of these carries source data that the output graph will not contain. Report
them under SOURCE COVERAGE, naming the ones that matter and what should be added to
cover them, unless the data is genuinely represented some other way.
```

Baseline: The prompts of the baseline mirror those of our approach. They contain the same task description, the same design steps, the review checklist of the review agent as a final self-check, and the same task-specific rules, adapted to an N-Triples file instead of a SPARQL endpoint and to writing the queries into a file instead of submitting them.

LLM judge: The judge receives the following instructions as its prompt in OpenCode. Only the configuration block at the top differs between runs.

````
# CONSTRUCT Query Evaluation: Judge Instructions

## Configuration

Wherever a name in braces such as `{KG_LABEL}` appears in these instructions, use the
value given here.

```
KG_LABEL          = <run label>
GENERIC_KG        = <QLever endpoint of G_D>
GENERIC_KG_FILE   = <path to G_D (.nt)>
CONSTRUCT_QUERIES = <path to Q>
OUTPUT_KG         = <path to G (.nt)>
ACCESS_TOKEN      = <access token>
OUTPUT_DIR        = <output directory>
GENERIC_KG_CONVENTIONS =
    Each row is a node <base/table/pk> (rows without a primary key: _rowN; composite
    keys: col=val pairs joined by ";"), typed with a class named after its table, with
    one triple per non-null cell using a predicate named after table and column;
    values carry XSD datatypes only where the source declares a column type and the
    value matches it, otherwise plain strings. Declared foreign keys are already edges
    to the referenced row (sometimes also kept as a literal), and if the schema is
    known, classes and the rdfs:domain/rdfs:range of predicates are declared too.
```

---

You are an expert in knowledge graph design, reviewing a set of SPARQL CONSTRUCT
queries **Q**. Q transforms a generic RDF graph **G_D**(which mirrors a source
dataset **D** record by record) into a semantic knowledge graph. You judge the semantic
modeling that Q performs, by reading Q and looking at the data in G_D. You also have
the resulting graph G, but only to verify issues you find in Q (rule 7 in section 2).

Review Q the way an experienced knowledge engineer would. Explore G_D as much as you
need to understand the domain, form your own view of how this data should be
modeled, and find the problems in Q. The criteria below tell you *what* to judge,
not what to look for: the considerations listed under each one are starting points,
not a checklist. Report any problem you find that falls under a criterion, including
kinds of problems not mentioned here.

---

## 1. Inputs

| Name | Variable | Description |
````

```
|---|---|---|
| Label | `{KG_LABEL}` | Name of the source dataset being modeled. |
| Generic KG (G_D) endpoint | `{GENERIC_KG}` | QLever endpoint URL of the generic input graph (e.g.
`http://localhost:9001`), or `NONE`. |
| Generic KG (G_D) file | `{GENERIC_KG_FILE}` | Path to the `.nt` file of the generic input graph, or
`NONE`. |
| CONSTRUCT queries (Q) | `{CONSTRUCT_QUERIES}` | File path(s) of the CONSTRUCT queries being judged. |
| Output KG (G) | `{OUTPUT_KG}` | Path to the `.nt` file of the output graph that Q produced from G_D. |
| Access token | `{ACCESS_TOKEN}` | QLever access token for both endpoints. |
| Output directory | `{OUTPUT_DIR}` | Directory where you write the result files. |

**Access to G_D.** You are given G_D as an endpoint, as a file, or both. Use whichever
suits each task. The endpoint is best for anything that counts, joins or compares
values across the graph. The file is handy for quick looks with `head`, `grep` or
similar, for example to see what raw triples look like or to find a specific value;
if you are given only the file, you can also load it with Python `rdflib` or
similar libraries and run SPARQL locally or use it in other ways as needed.
Ideally never print the whole file.
For anything large, prefer the endpoint. Record file commands in the query log as
evidence, like queries.

**Querying an endpoint.** G (and G_D, if given as an endpoint) is served by QLever. Send each
query as an HTTP GET request to the endpoint URL, with the query and the access token
`{ACCESS_TOKEN}` as URL parameters and `Accept: application/sparql-results+json`.
Results are in `results.bindings`. For example, in Python:

```python
import requests

def select(endpoint, sparql):
    r = requests.get(endpoint,
                     params={"query": sparql, "access-token": "{ACCESS_TOKEN}"},
                     headers={"Accept": "application/sparql-results+json"},
                     timeout=120)
    r.raise_for_status()
    return [{k: v["value"] for k, v in row.items()}
            for row in r.json()["results"]["bindings"]]

rows = select("{GENERIC_KG}", "SELECT ?p (COUNT(*) AS ?n) WHERE { ?s ?p ?o } GROUP BY ?p")
```

Declare every prefix you use in each query; QLever rejects undeclared prefixes. If a
request fails, check the error text in the response body before retrying.

Structure of G_D. G_D mirrors the source structure: each source record (a row, a
JSON object, etc.) is one node and each source field is a predicate. Note what G_D
already provides (for example types, datatypes or links between records), so that you
judge what Q adds or loses relative to it, not what the converter already did.
Conventions of this G_D: {GENERIC_KG_CONVENTIONS}

---

2. Ground rules

1. Use only the inputs above. Do not open other files in the working directory.
2. Do not execute the CONSTRUCT queries. You may run any read-only SELECT queries
   against G_D and G.
3. Every issue you report needs evidence: a quoted fragment of Q, or a query on G_D
   and its result. Do not report problems based on what the data "probably"
   contains; check it.
4. Record every query you run on G_D or G, numbered, stating which graph it ran on, with a one-line
summary of its
   result, in `{OUTPUT_DIR}/{KG_LABEL}_queries.md`.
5. When you look at samples of G_D, order them with `ORDER BY STR(?x)` before
   `LIMIT`, so that repeated reviews see the same values.
6. Print short progress notes as you go, so the transcript is readable.
7. The output KG G is provided only to verify issues you find in Q. Use it to confirm
   or rule out a suspected problem (for example, whether references actually match
   the IRIs of the entities they point to, or whether a cast actually lost values).
   Do not judge G's size, completeness or data quality on its own, and do not report
   issues that you cannot trace back to a specific part of Q.
```

```
---

## 3. How to work

1. **Understand D.** Explore G_D: its sources, fields, values, and how records refer
   to each other. Decide what the domain is and what the real-world things and
   relationships in it are.
2. **Understand Q.** Read Q in full. Work out which entities it creates, how it mints
   their IRIs, and which triples it produces from which fields.
3. **Review.** Go through the criteria in section 4. For each, compare what Q does
   with how you think the data should be modeled, and record every issue you find.
4. **Score.** Assign each criterion a level using section 5.

---

## 4. Criteria

**C1. Entity and class modeling.** Do the right real-world things become entities,
at the right granularity? Examples of problems (not a complete list): things in D with their own
identity that never become entities; records that only connect other records turned
into entities; distinct kinds of things merged into one class, or one kind split
into several.

**C2. Vocabulary.** Do class and predicate names denote domain concepts, rather than
repeating names from the input generic KG? Does each predicate have one precise meaning and a natural
direction? Examples of problems (not a complete list): names that cannot be understood without the
source; predicates reused for different meanings; triples that read backwards;
inconsistent naming.

**C3. IRI design.** Are entity IRIs minted from stable identifiers of D, in a
namespace that reflects the domain? Examples of problems (not a complete list): row positions,
random non-deterministic values values used as IRI IDs; incorrigible source names in IRI paths;
values placed in IRIs without the encoding they need.

**C4. Relationships.** Are the relationships of D represented as links between
entities, faithfully without G being a structural replica of the source schema? Examples of problems
(not a complete list): references kept as literal IDs;
links built from the wrong fields or joining the wrong records; relationships that
carry their own information flattened so that information is lost.

**C5. Self description.** Does every entity get an `rdf:type` and relevant entities get a human-readable
`rdfs:label`? Examples of problems (not a complete list): classes without types or labels; labels
built from fields that are often missing; identifiers used as labels where a name
exists. An entity with no natural name does not need a label.

**C6. Schema coverage.** Does the information of every field in D reach the semantic
graph? A field used only in a filter or join condition does not carry its
information over by itself. Leaving out purely technical fields is fine.

**C7. Soundness.** Does Q produce only content supported by D? Examples of problems (not a complete
list): fixed facts written onto entities; links D does not state; missing
values filled in with invented defaults. Values computed purely from D are fine.

**C8. IRI consistency.** Does the same real-world entity always get the same IRI,
wherever Q mints or references it? Compare the IRI expressions used in different
places, and check with real values from G_D that references would produce the same
IRI as the entity they point to.

**C9. Literal typing.** Do literals get datatypes that match their values? Look at
real values in G_D, not just at field names. Keep in mind that in SPARQL a failed
cast leaves the variable unbound, so the value silently disappears.

---

## 5. Severity and levels

**Severity.** Decide the severity of each issue yourself, using this principle:

- **MAJOR**: the issue loses or corrupts information, makes the graph misleading, or
  makes an important part of the domain wrong or hard to query. Issues that affect a
```

```
  main class, a main relationship, or a large share of the data are usually major.
- **MINOR**: the information is still there and correct, but it is less clear, less
  consistent, or less convenient than it should be, or the issue affects only a few
  values or a marginal part of the domain.

Explain each severity decision in one sentence.

**Levels.** Give each criterion one level:

| Level | Anchor |
|---|---|
| 1 | No issues. |
| 0.75 | Only minor issues. |
| 0.5 | One major issue, or minor issues so widespread that they affect most of the graph. |
| 0.25 | Several major issues, but the criterion is still partly met. |
| 0 | The criterion is essentially not met. |

Explain in one or two sentences why you chose this level and not a neighboring one.

**Total.** Sum the 9 levels for a score out of 9, and report it scaled to 10:
`(sum / 9) × 10`, rounded to 1 decimal.

**Other observations.** If you find problems in Q that do not fit any of the nine
criteria, report them separately. They are not scored.

---

## 6. Outputs

Write these files to `{OUTPUT_DIR}`.

**`{KG_LABEL}_results.json`**

```json
{
  "kg_label": "{KG_LABEL}",
  "domain_summary": "2-4 sentences: what D is about and how you think it should be modeled",
  "criteria": {
    "C1": {
      "issues": [
        {
          "severity": "MAJOR",
          "description": "...",
          "severity_reason": "...",
          "evidence": "Q fragment: ... / query #7"
        }
      ],
      "level": 0.75,
      "rationale": "..."
    }
  },
  "other_observations": [
    {"description": "...", "evidence": "..."}
  ],
  "sum_out_of_9": 0.0,
  "score_out_of_10": 0.0
}
```

Include all nine criteria, C1 to C9. A criterion with no issues has an empty
`issues` list and level 1.

`{KG_LABEL}_queries.md`: every query and file command you ran on G_D or G,
numbered, each with the graph it ran on and a one-line result summary.

When you are done, print a table in the conversation with the 9 levels, the sum out
of 9 and the score out of 10, followed by the three most important issues you found,
each in one sentence.
```